

Answers to Leant2's 31 Expert Questions

From improvement proposals to a
trustworthy synthesis architecture

Prepared for Vladimir Reshetnikov

21 September 2026

Reviewed project	https://github.com/VladimirReshetnikov/Leant2
Proposal	docs/proposals/11-further-improvements/
Pinned commit	784664ff34e819422510a4d470ab07fe48bb736b
Pinned toolchain	<code>leanprover/lean4:v4.34.0</code>
Method	Source inspection, primary-literature investigation, explicit mathematical arguments, and independently executed finite Python checks.

Abstract. This report answers every question in the nine “Questions for experts” groups of Leant2’s further-improvements proposal. The central recommendation is to separate five guarantees that the proposal sometimes places too close together: kernel-certified acceptance, conservative pruning, completeness relative to an explicitly bounded language, optimality for a frozen ranking objective, and reproducibility of a logical execution prefix. Each requires different evidence.

The investigation identifies useful existing work that changes several starting premises. Smyth already addresses incomplete recursive examples and implements higher-order examples; Hoogse+ already supports type classes through dictionary passing; Canonical handles important indexed-recursion examples; AutoLifter directly informs auxiliary-state synthesis; and the pinned Lean release exposes both motive-construction functions and proof-producing call-by-value evaluation. None of these results, alone, solves Leant2’s complete problem.

The proposed architecture combines dependency-aware persistent search states, certified residual observations, constrained carrier and motive grammars, a feature-separation-and-closure procedure for carrier proposals, capability-based recursive bodies, local program–proof obligations, abstraction-refined retrieval, and auditable optional learned guidance. Formal statements clarify which pruning and completeness claims are justified. A small exhaustive experiment demonstrates an actual four-state lower bound from nine observations and checks several counterexamples used in the analysis.

Evidence boundary. No Leant2 build, Lean compilation, model evaluation, or reproduction of the repository’s performance measurements was performed. Implementation sketches are designs, not tested patches. The included Python checks were executed and their results are recorded. Mathematical arguments in this article are ordinary proofs, not Lean formalizations.

Contents

1	Scope, findings, and how to read this report	3
1.1	The exact material investigated	3
1.2	The most consequential corrections	3
1.3	What to implement first	4
2	Five guarantees and a common semantic model	4
2.1	Acceptance is only one of the guarantees	4
2.2	Partial states and completion sets	5
2.3	A result algebra that does not overclaim	5
3	R1: Search control with coupled goals	6
3.1	R1.1. Can shared goals be organized without copying everything or serializing everything?	6
3.2	R1.2. What admissible heuristic survives unification between goals?	7
3.3	R1.3. Can just-in-time learning coexist with principled ranking?	8
3.4	R1.4. Can strict wall-clock budgets produce machine-independent results?	8
4	R2: Carriers, accumulators, and auxiliary state	9
4.1	R2.1. Can finite examples construct a minimal carrier type or prove none exists?	9
4.2	R2.2. What is the alphabet for polymorphic carrier learning?	11
4.3	R2.3. Can backward fusion discover auxiliaries from examples?	12
4.4	R2.4. Is there a complete, practical motive grammar for primitive recursion with parameters?	13
4.5	R2.5. How do induction generalization and lemma discovery transfer?	14
5	R3: Evaluation of programs with holes	14
5.1	R3.1. How far do live bidirectional evaluation and higher-order examples extend?	14
5.2	R3.2. What incremental normalizer is suitable for assignment and rollback?	15
5.3	R3.3. What is the cheapest route to a fast conservative evaluator?	16
5.4	R3.4. Can top-down angelic search have termination and completeness guarantees?	17
6	R4: Indexed families and motives	18
6.1	R4.1. Is occurrence-based motive enumeration complete and closed under accumulation?	18
6.2	R4.2. How should search represent transports without losing canonicity?	19
6.3	R4.3. What unfolding policy is supported by experience in existing search?	19
6.4	R4.4. Can Lean’s motive computation be called directly and diagnosed?	20

7	R5: Recursion from incomplete examples	21
7.1	R5.1. Does a capability interpreter agree definitionally with the realized program?	21
7.2	R5.2. What does top-down angelic recursion gain and lose?	22
7.3	R5.3. Is there a canonical small basis of recursion schemes with measured coverage?	22
8	R6: Contracts beyond decidable observations	23
8.1	R6.1. How should value holes and proof holes be interleaved?	23
8.2	R6.2. Can the easy induction–simplification–arithmetic class be recognized?	24
8.3	R6.3. Which automated-induction facilities are usable now?	25
8.4	R6.4. What scheduling policy should pay proof debt?	25
9	R7: Component retrieval at library scale	26
9.1	R7.1. What abstraction is useful and sound for dependent signatures?	26
9.2	R7.2. Has abstraction refinement already been combined with type classes?	27
9.3	R7.3. Can premise selection serve data goals and example-based retrieval?	27
10	R8: Ranking and candidate equivalence	28
10.1	R8.1. What objective makes ranking defensible?	28
10.2	R8.2. Which equality should group dependent candidates?	29
11	R9: Learned guidance under a kernel gate	29
11.1	R9.1. How accurate are models at proposing carriers or helpers from types and examples?	29
11.2	R9.2. What makes learned contributions deterministic and auditable?	30
12	Integration plan and acceptance gates	31
12.1	One architecture, not nine independent add-ons	31
12.2	A concrete end-to-end search loop	33
12.3	What would justify a negative or optimality claim?	33
13	Evaluation protocol and results actually obtained	34
13.1	Separate expressibility, discovery, and certification	34
13.2	Adversarial regression cases	34
13.3	Executed finite-model checks	34
13.4	Unmeasured outcomes that remain open	35
14	Conclusion	35
A	Question-to-answer coverage map	36
B	Reproduction and source notes	38
	References	39

1 Scope, findings, and how to read this report

1.1 The exact material investigated

The target is the proposal at commit `784664ff34e819422510a4d470ab07fe48bb736b`. Its eight question-bearing source files contain nine research themes: R1 through R7 have one file each; R8 and R9 share `sections/12-ranking.tex`. There are exactly 31 numbered expert questions: $4 + 5 + 4 + 4 + 3 + 4 + 3 + 2 + 2$. Sections 3–11 preserve these identifiers, while Appendix A provides a coverage map. Questions are paraphrased rather than reproduced wholesale. The proposal and implementation are cited separately throughout.[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

The implementation inspection covered the acceptance gate, the relevant search and residual-evaluation code, the engine’s enumeration and ranking logic, and the toolchain declaration. This is a targeted architectural review, not a security audit of every repository file. The original proposal explicitly places its measurements at an earlier commit, `33cec8b`, on one Windows workstation. They are not measurements of the designs proposed here and were not reproduced in this investigation.[2, 11, 12, 13, 14]

The article distinguishes *established results*, with primary-source citations; *derived conclusions*, justified by arguments given here; *proposed mechanisms*, requiring implementation and evaluation; and *unmeasured questions*, for which a protocol is specified instead of an invented answer. An elementary lemma proved here is not thereby claimed to be a new result in the literature.

1.2 The most consequential corrections

Starting premise	Investigation	Consequence for Leant2
One unit per open goal is an admissible lower bound.	Shared assignments can close several obligations; some closures cost zero.	Start with $h = 0$ or a maximum of proved relaxed bounds, not an unqualified sum. See R1.2.
Finite samples determine a minimal carrier type.	They constrain compatible implementations, not a unique intended function or type representation.	State the grammar, cost, and semantic bounds; return only bounded minimality. See R2.1.
Observed constructible values bound a carrier’s capacity.	An enumerated set is ordinarily a lower bound on available values, not an upper bound on reachable states.	Use certified cardinality or reachable-state invariants for hard pruning.
Reverse needs a larger semantic carrier.	Reverse is a right fold into lists when the step may append.	Continuation carriers solve a representation/search/cost problem, not an existence obstruction.
Smyth is confined to trace-complete, first-order examples.	Its formal core is restricted, but the system removes trace-completeness requirements and implements higher-order examples.[23]	Adapt the actual system’s constraint ideas; do not rebuild from a weaker premise.
Carrier invention has no closely related synthesis algorithm.	AutoLifter synthesizes lifting auxiliaries under algorithmic schemes, with assumptions different from Leant2’s.[21]	Reuse distinguishability and closure ideas, while retaining backtracking and explicit bounds.

Starting premise	Investigation	Consequence for Leant2
Indexed recursor synthesis is unaddressed by existing search. TYGAR has not treated type classes.	Canonical demonstrates indexed structural recursion and uses suitable generalized induction principles.[16] Hoogse+ represents them by dictionary passing and handles instance components.[34]	Separate direct motive inference from invention of a useful generalized principle. The new work is Lean dependency, universes, local instances, and scale—not dictionary passing itself.
A new evaluator must precede proof-producing evaluation.	Lean 4.34.0 includes <code>cbv</code> and <code>decide_cbv</code> ; its motive helpers are also public.[29, 28]	Evaluate these APIs before implementing another evaluator or motive elaborator.
Reordering preserves bounded search outcomes.	It preserves a finite alternative set only when that entire set is explored; a deadline may expose a different subset.	Keep timeout, exhaustion, optimality, and replay claims separate.

These corrections do not make the proposal’s research agenda disappear. They make its boundaries more precise. In particular, a finite passive sample still does not determine a general dependent carrier; a higher-order sketch language is not a verified evaluator for Lean’s full conversion relation; and a successful indexed example is not an algorithm for all generalizations.

1.3 What to implement first

The highest-value first change is a correctness-oriented state and observation interface, not a larger collection of synthesis rules. Make branch state explicit, record why each observation was decided or suspended, and separate proposals from evidence. Then remove narrowly identified implementation restrictions: allow contextual proof obligations, admit a controlled indexed-induction path, and test the pinned proof-producing evaluator. Only after those foundations should carrier invention, library-scale retrieval, and learned ranking compete for the budget. The staged plan in Section 12 gives concrete gates rather than estimated completion times.

2 Five guarantees and a common semantic model

2.1 Acceptance is only one of the guarantees

Fix a Lean environment E , a local telescope Γ , a target type T , a contract $C : T \rightarrow \mathbf{Prop}$, and an allowed-axiom profile A . A certified result is a term p and evidence π such that, after the required abstraction of locals and universe handling,

$$E \vdash p : T, \quad E \vdash \pi : C(p), \quad \text{Axioms}(p, T, \pi, C(p)) \subseteq A.$$

These judgments say nothing, by themselves, about whether other valid programs were missed, whether p is the preferred answer, or whether its compiled execution has a particular performance profile.

The current gate is a strong foundation: it rejects unresolved holes and escaped locals, generalizes universe metavariables jointly, performs synchronous checked declaration insertion, and audits the transitive axiom closure. The engine constructs the contract application for the candidate

before calling the gate.[11, 13] A useful API hardening is to make the gate own a frozen query and reconstruct the required contract internally. This reduces the chance that a future provider supplies a proof of the wrong proposition. It is not a claim that the inspected engine currently does so.

Definition 2.1 (The five guarantees). *Acceptance soundness* means that every certified result satisfies the exact advertised query and axiom policy. *Pruning conservativity* means that a discarded partial state has no permitted successful completion. *Relative completeness* means that every successful program in a stated bounded language remains discoverable by an exhaustive, fair run. *Ranking optimality* means that no eligible program has a better value of one fixed objective than the returned optimum. *Replay determinism* means that fixed inputs and a fixed logical execution policy reproduce a specified execution prefix and its outputs.

An unsound pruning heuristic can leave acceptance soundness intact: every surviving answer still checks, but the only valid answer may have been discarded. A time-limited fair enumerator may be sound without completing its finite search space. A dynamically learned search order can improve success rates while destroying an A* optimality argument. These are not terminological details; they determine which user-visible conclusions are legitimate.

2.2 Partial states and completion sets

Let G be a fixed grammar of program, type, recursion, and proof-plan productions, with explicit bounds B . A partial state can be described schematically by

$$S = (E, \Gamma, \Sigma, U, D, O, Q, g, \rho),$$

where Σ is the expression-metavariable assignment state, U contains universe constraints, D is the dependency structure, O contains residual observations and their evidence, Q is the collection of typed obligations, g is the accumulated frozen search cost, and ρ records branch identity and policy versions. A real implementation must also account for delayed assignments, local instance context, transparency settings, and the state of any elaborator service it invokes.

Write $\text{Comp}_{G,B}(S)$ for the well-typed completions admitted by the grammar and bounds, and

$$\text{Sol}(S) = \{p \in \text{Comp}_{G,B}(S) \mid E \vdash C(p) \text{ with permitted evidence}\}.$$

A hard refuter must justify $\text{Sol}(S) = \emptyset$. A heuristic feature such as “this branch passed more examples” merely changes a priority. The implementation should make it difficult to confuse these two interfaces.

```
Proposal:   State -> ordered alternatives + heuristic features
Observation: State -> proved | refuted | suspended(dependencies)
Certificate checker: FrozenQuery -> Candidate -> certified | rejected
Scheduler:  frontier + immutable policy epoch -> next event
```

This is an interface sketch, not Lean code. In particular, the observation constructor `refuted` should carry a checked reason or be produced only by a component with an established conservative semantics.

2.3 A result algebra that does not overclaim

A useful public result distinguishes certified solutions, a certified impossibility theorem, finite-grammar exhaustion, and an interrupted or unresolved search. Finite-grammar exhaustion

needs the grammar identity, bounds, and coverage record; it is not a proof that the target is uninhabited. A candidate that passes tests but lacks a proof belongs in a separate exploratory result, not in the certified list.

Even finite syntax does not make every operational issue disappear. Literal magnitudes, universe expressions, auxiliary declarations, and proof plans must be bounded or effectively enumerated. A timed-out type check or unresolved proof attempt is *unknown*, not a failed candidate. An exhaustive-syntax receipt cannot honestly claim semantic exhaustion if some of its members remain unresolved. Finally, a universe-specialized candidate must be labeled with its specialized type; success at a **Prop** instantiation is not success at an arbitrary original universe. The inspected engine already stores the accepted program type, and a receipt should preserve that distinction.[13]

3 R1: Search control with coupled goals

3.1 R1.1. Can shared goals be organized without copying everything or serializing everything?

Answer. Yes, in the useful sense of exploiting *proved noninterference* and persistent sharing. No, in the stronger sense of treating genuinely coupled alternatives as independent without retaining their correlations. Use a dynamic dependency hypergraph and memoize checked solution schemes, not raw goal identifiers.

Aesop is directly relevant because its search architecture explicitly handles metavariable dependencies. Its general best-first organization should be studied as an implementation reference rather than cited as a theorem that every dependent search is optimally factorizable.[15] Leant2's current continuation discipline is already correct about joint assignments: a branch succeeds only after its children and the remaining obligations succeed together.[12] Refactoring must preserve that property before changing priorities.

For each obligation q , compute an over-approximation $R(q)$ of the expression and universe variables it can read, including variables reachable through its target, local declaration types and values, delayed assignments, and relevant instance constraints. Rule applications also have a write set $W(q)$. Disjoint visible metavariables in two displayed target types are insufficient: a metavariable hidden in a local's type or a pending instance constraint can couple the goals.

A sufficient condition for independent expansion of components I and J is

$$W(I) \cap (R(J) \cup W(J)) = \emptyset, \quad W(J) \cap (R(I) \cup W(I)) = \emptyset,$$

together with immutable shared environment and policy. Dependencies must be recomputed or conservatively updated when unification changes the graph. Shared universes and unresolved class arguments should initially force components together rather than be ignored for speed.

Proposition 3.1 (Independent replay). *If two state transitions have disjoint read/write interference as above, introduce fresh identifiers disjointly, and do not alter shared environment or policy, their assignments commute up to renaming of fresh identifiers.*

Proof. Each transition reads the same pre-existing values regardless of whether the other has run. Neither overwrites a value written or read by the other. Their resulting substitutions can therefore be united in either order. Fresh identifiers are compared modulo the stipulated renaming. This statement does not apply to transitions whose hidden elaborator effects have not been included in their read/write sets. $\square$

Operationally, keep persistent branch snapshots and a queue of connected components, but join components whenever a new dependency is introduced. This avoids deep physical copies of immutable structures while retaining separate assignment histories where necessary. A shared-constraint separator can support more elaborate AND/OR factorization, but the cached object is then conditional on the separator assignment; it is not an unconditional proof of the subgoal.

For cross-branch memoization, abstract a solution over the minimal dependency-closed local telescope, freshen its universes, instantiate it in the new context, and recheck its type. A successful cached term can be useful even when a search-state snapshot cannot be transplanted. Negative memoization is more delicate: a failed unification, timeout, or bounded proof attempt must not be reused as global unsatisfiability. Cache such events only as scheduling information unless their scope and exhaustive bounds match exactly.

3.2 R1.2. What admissible heuristic survives unification between goals?

Answer. The universally safe starting point is zero. A useful next step is the *maximum* of justified relaxed-obligation bounds; an additive bound needs a separate cost-partition argument.

The proposed “one unit per non-leaf goal” can overestimate. Suppose two outstanding constraints refer to the same metavariable and the single cost-one assignment $?x := 0$ closes both. Counting the two constraints yields two although the remaining cost is one. Zero-cost exact rules provide an additional obstruction. The included Python check records this simple countermodel; it is a mathematical objection to the proposed bound, not a measured Leant2 failure.

Proposition 3.2 (Safe maximum of projections). *For each obligation q_i , suppose every complete continuation r of state S induces a solution of a relaxation $\mathcal{R}_i(S)$ of cost at most $\text{cost}(r)$. If $h_i(S)$ lower-bounds the optimum of that relaxation, then*

$$h(S) = \max_i h_i(S)$$

is an admissible lower bound on the cost of a complete continuation.

Proof. For every complete continuation r and every i , the assumptions give $h_i(S) \leq \text{cost}(r)$. Taking the maximum preserves the inequality; taking the infimum over complete continuations proves the claim. $\square$

One implementable relaxation erases difficult term indices while retaining available heads and typed production costs. It must over-approximate possible concrete actions, including actions that solve several goals at once. Another uses a hypergraph of required constructor or provider actions. Both require testing the projection property; a type-head distance that forgets an available zero-cost local is not admissible.

A sum is valid when costs can be partitioned into disjoint responsibilities and each concrete continuation pays every charged responsibility. Merely putting goals into distinct list entries is not such a partition. Independent components from R1.1 may support additive costs, but only if the cost model excludes a shared action that closes both or pays for shared work once.

Finally, admissibility is relative to an objective. A* over rule costs does not prove optimality for the engine’s current tuple of unused inputs, eliminator count, and raw term size.[13] Normalization can erase previously counted syntax, and later filling a hole can make an input used. Use either a deliberately additive derivation objective, or a lower bound designed for the actual final ranking. Zero-cost cycles must be eliminated or bounded; otherwise even a zero heuristic need not yield a terminating finite-cost layer.

3.3 R1.3. Can just-in-time learning coexist with principled ranking?

Answer. It can coexist with acceptance soundness and eventual fair enumeration. It does not automatically preserve the same deadline-limited answers or a frozen optimality certificate.

Probe gives a concrete precedent for learning enumeration priorities from partially successful programs during synthesis.[17] Its relevance is the learning signal and enumeration mechanism, not a guarantee that two time-limited runs expose the same candidates or that their first result matches user intent.

Leant2 should maintain two distinct numerical quantities. The first is a *certificate-facing cost*, fixed for a policy epoch and used for lower bounds, exhaustion layers, and any optimality statement. The second is a *dispatch priority*, which may incorporate observations survived, model scores, measured evaluation cost, and information expected from an assignment. Learning changes the second without retroactively changing the first.

A simple policy alternates a deterministic base queue with a learned queue using fixed logical quanta. Every admitted alternative remains in the base queue. Learned priorities are frozen between explicit epochs; an epoch change records its training evidence and new weights. This is less aggressive than updating a probability at every node, but it is easier to reproduce and evaluate. A more dynamic policy can be valid, yet it must update stale queue keys and invalidate any bound whose costs changed.

The claim “only reordering, therefore complete up to the bound” needs qualification. It is correct after exhaustive exploration of a fixed finite alternative set, under fair scheduling and conservative pruning. It is false when “bound” means ten seconds: reordering changes which alternatives receive those seconds. The receipt should say “best certified result found in this logical prefix,” unless a separate frontier certificate proves more.

User-perceived arbitrariness should be measured, not settled by an algorithm name. Record rank stability under irrelevant renaming, harmless provider-order permutations, repeated runs, and new examples. Compare fixed-policy and learned-policy results with the same semantic candidate objective and the same logical budget as well as the same wall-clock budget.

3.4 R1.4. Can strict wall-clock budgets produce machine-independent results?

Answer. Not simultaneously with unrestricted machine-speed variation and a requirement to return every newly discovered useful result. The achievable invariant is the result for a fixed logical prefix, not identical progress by a fixed deadline.

Proposition 3.3 (Deadline obstruction). *Consider a progress-sensitive search that may first certify a new answer after an indivisible computation of positive cost. Without a lower bound on machine speed, no fixed wall-clock deadline guarantees both execution of that computation and identical progress-sensitive output on all machines.*

Proof. Choose a fast execution in which the computation completes before the deadline. Arbitrarily slow the same computation so it does not complete before that deadline. The slow execution cannot report its unchecked result as certified. If the fast execution reports it and the slow one does not, outputs differ. Suppressing the fast result or abandoning the deadline removes one of the stated requirements, rather than resolving the conflict. $\square$

This does not rule out reproducible products. Offer a deterministic logical work policy—fixed production order, stable tie breakers, explicit reduction/proof quanta, and recorded proposal

batches—with a separate wall-clock cancellation mechanism. Completed logical prefixes replay identically. A slow machine may report an earlier prefix or an incomplete checkpoint. Calibrating node counts reduces variance but cannot account for every query-dependent reduction or instance-search cost.

Heartbeats and node counters are useful resource measures, not hard real-time guarantees. In the inspected engine, deadlines are checked around search work while individual reductions, elaborator calls, or checks may have their own behavior; the query context also disables the general heartbeat limit in part of the pipeline.[13, 12] Bounded tactic calls and an external worker cancellation boundary are preferable to assuming that every expensive operation will reach the next node check promptly.

The Luby restart theorem concerns Las Vegas algorithms under its runtime-distribution assumptions.[18] It does not establish an optimal allocation among dependent, learned, resumed Lean search lanes. Keep it as motivation for a restart baseline, not as the proof of the chosen portfolio. Measure resumed versus restarted work, including the cost of reconstructing contexts and caches.

4 R2: Carriers, accumulators, and auxiliary state

4.1 R2.1. Can finite examples construct a minimal carrier type or prove none exists?

Answer. They can support exact minimality within an explicitly finite, decidable checked hypothesis space. They cannot, without further assumptions, identify the globally intended function, its unique minimal representation, or absence of an implementation in an unbounded dependent grammar.

The classical fold characterization concerns a *given function*: its kernel must respect the input constructor if it is to be realized directly as a fold.[19] Finite examples instead describe a set of possible functions. Even the word “minimal” is ambiguous: fewest semantic states, smallest carrier-type syntax, shortest program, cheapest execution, and easiest proof can select different answers. A carrier such as `Nat` has short type syntax and infinitely many values.

Fix a grammar of carrier types and bounded grammars of seeds, step functions, finishers, and relevant proofs. Bound literal encodings and universe expressions as well. If every resulting candidate can be completely checked against the intended finite observations, exhaustive enumeration in a fixed total cost order returns a minimum-cost consistent hypothesis or establishes that none exists *in that finite space*. If checking a candidate is merely timed out, the conclusion is incomplete, not “none.” For a universal contract, finite observations alone cannot discharge the contract.

The right semantic lower bound

Let A be an alphabet and $h : A^* \rightarrow Y$ a specified output function. A right-fold implementation with hidden carrier K consists of

$$s(\epsilon) = z, \quad s(ax) = \text{step}(a, s(x)), \quad h(x) = \text{finish}(s(x)).$$

Define the residual relation

$$x \sim_h y \iff \forall u \in A^*, h(ux) = h(uy).$$

It is stable under prepending a symbol. A carrier state must distinguish suffixes that some common prefix distinguishes. This is a residual-state viewpoint analogous to the automata-learning setting, but passive samples do not supply the membership and equivalence oracles used by active learning.[20]

Proposition 4.1 (State separation). *For any implementation above, $s(x) = s(y)$ implies $x \sim_h y$. Consequently, if a finite observation set supplies $h(ux) \neq h(uy)$ for one common prefix u , then every consistent implementation has $s(x) \neq s(y)$.*

Proof. Induction on the prefix, using the same deterministic step function, shows $s(ux) = s(uy)$ whenever $s(x) = s(y)$. Applying the common finisher gives $h(ux) = h(uy)$. The contrapositive proves the consequence. $\square$

Construct a finite *incompatibility graph*: vertices are observed suffixes and an edge joins x, y when a common observed prefix forces unequal outputs. Any consistent assignment of carrier states is a proper coloring. Thus the graph’s chromatic number is a lower bound on the number of reachable states. A clique gives a cheaper certified lower bound without solving graph coloring optimally.

This justifies refuting Bool when three or more pairwise incompatible states are forced. It does *not* justify refuting Nat because only two natural values happened to be enumerated so far. The latter set is not an upper bound on what a step can produce. A bounded-image argument is sound only when a separate invariant proves that every allowed implementation remains in that image.

A worked four-state example

Let $h(x)$ say that a word over $\{a, b\}$ contains both symbols. Use suffixes ϵ, a, b, ab and prefix contexts ϵ, a, b :

suffix x	$h(x)$	$h(ax)$	$h(bx)$
ϵ	0	0	0
a	0	0	1
b	0	1	0
ab	1	1	1

The rows are pairwise different, so the incompatibility graph is K_4 . The table uses only nine distinct input words, because some concatenations coincide. Four states are necessary. They are also sufficient: store two bits, “seen a ” and “seen b ,” update the corresponding bit on each step, and finish with conjunction. An induction proves that the bits have exactly their advertised meanings after every suffix.

The accompanying script accounts exhaustively for all deterministic right-fold machines with one, two, or three labeled states, including every seed, every two-letter transition table, and every Boolean finisher. There are respectively 2, 128, and 17,496 such machines; none fits the nine samples. The four-state witness was additionally checked on all 511 words of length at most eight. The finite experiment corroborates the example; the universal sufficiency claim follows from the invariant proof, not from the 511 tests.

Two traps in using observation tables

First, partial-row compatibility is not an equivalence relation. The rows $\{c \mapsto 0\}$, $\emptyset$, and $\{c \mapsto 1\}$ are compatible first-to-second and second-to-third, but not first-to-third. Unknown

entries must not be filled by arbitrary defaults or transitively merged into one state. Maintain constraints or alternative partitions.

Second, reverse is not a counterexample to existence of a list-valued right fold:

$$\text{reverse}(xs) = \text{foldr}(\lambda a \ r. r ++ [a]) [] xs.$$

An endofunction carrier can make accumulation efficient and make the step expressible in a much smaller search grammar. Failure of an append-free or shallow step grammar, however, is not failure of the fold characterization. Report representation restrictions explicitly.

4.2 R2.2. What is the alphabet for polymorphic carrier learning?

Answer. Positions are justified for a genuinely parametric list transformer, one length at a time. Equality patterns require additional operations or assumptions; a type parameter alone does not grant an equality test. Arbitrary Lean inhabitants do not automatically come with the needed naturality theorem.

Here is the precise useful statement. Suppose $f_A : \text{List}(A) \rightarrow \text{List}(A)$ is a family satisfying

$$f_B(\text{map}(g, xs)) = \text{map}(g, f_A(xs)) \quad (g : A \rightarrow B). \quad (1)$$

For each n , let $t_n = [0, \dots, n-1]$ be the list of all positions in $\text{Fin}(n)$.

Proposition 4.2 (Positional representation). *Under (1), for every list xs of length n ,*

$$f_A(xs) = \text{map}(\text{lookup}_{xs}, f_{\text{Fin}(n)}(t_n)).$$

Proof. By definition, $xs = \text{map}(\text{lookup}_{xs}, t_n)$. Substitute this equality into (1), with the source type $\text{Fin}(n)$ and target type A . The case $n = 0$ is included: lookup is the unique function from the empty position type. $\square$

The theorem permits duplication, deletion, and reordering of positions; it does not assert a uniform finite-state bound across all lengths. A sequence of positional behaviors for lengths $0, 1, 2, \dots$ can still be arbitrarily complicated. It also does not allow the program to invent an A value not represented by an input position.

For Leant2, make the parametricity assumption explicit in the generated fragment or supply a relational certificate for the candidate. Opaque element types, permitted constructors, and a whitelist of uniform operations offer a practical first fragment. Type-class dictionaries can deliberately reveal equality, order, or other structure; their behavior then belongs in the task signature. With lawful decidable equality and bounded input length, equality patterns can be represented by partitions of positions. Without such a dictionary, treating those partitions as observable invents power the program may not possess.

Conversely, classical or type-inspecting constructions need not satisfy the all-functions naturality equation. A polymorphic surface type is not, by itself, a Lean proof of (1). The operational proposal is therefore: use tagged positions to generate tests and features, but use them for exact pruning only within an established parametric fragment. Treat register/nominal automata as a possible later representation of unbounded data states, not as an immediate consequence of a finite positional sample.

4.3 R2.3. Can backward fusion discover auxiliaries from examples?

Answer. Yes as a constrained synthesis-and-refinement strategy; not as a unique inverse of fusion. AutoLifter is an important existing reference. A useful Leant2 adaptation is to separate *state distinguishability* from *closure under recursive update*.

AutoLifter starts with substantially more structure than a bare type and a handful of examples: an original computation, an algorithmic paradigm, and languages for the lifting auxiliary and combination function. Its decomposition exposes distinguishability requirements, but an auxiliary satisfying one subproblem may fail to support a later update. Its minimality statements should not be read as global minimum-cardinality carrier inference from passive examples.[21] This makes it relevant, rather than a direct solution to the whole question.

A concrete feature-separation-and-closure procedure

Choose a bounded library of typed features

$$\phi_j : A^* \rightarrow S_j.$$

Candidates may include emptiness, a last/first element with an option type, length modulo a small modulus, membership flags for concrete alphabets, or already available measures. A candidate feature must itself be expressible under the intended type and operation restrictions. A polymorphic element cannot be compared to a distinguished value that the context does not supply.

Each ϕ_j separates some edges of the incompatibility graph from R2.1. Solve a weighted set-cover problem to propose a small feature set J separating every observed edge. Form

$$K_J = \prod_{j \in J} S_j, \quad \phi_J(x) = (\phi_j(x))_{j \in J}.$$

Next synthesize z , step, and finish. The essential obligations are not merely sample agreement but the closure equations

$$\phi_J(\epsilon) = z, \tag{2}$$

$$\phi_J(ax) = \text{step}(a, \phi_J(x)), \tag{3}$$

$$h(x) = \text{finish}(\phi_J(x)). \tag{4}$$

When h is only partially specified, replace (4) by the actual contract and retain its proof obligation. Samples can suggest all these equations, but only proofs justify their universal use.

Proposition 4.3 (Feature realization). *If z , step, and ϕ_J satisfy the seed equation and (3) for all inputs, then*

$$\text{foldr}(\text{step}, z, x) = \phi_J(x).$$

If (4) also holds, the resulting fold implements h .

Proof. Induct on x . The empty case is the seed equation. In the constructor case substitute the induction hypothesis into the fold equation and apply (3). Applying the finisher gives the second conclusion. $\square$

A failed closure attempt is informative. Two suffixes can agree on all currently selected features but require different successor features after the same input constructor. Add that pair to a closure-incompatibility graph and propose another feature. If no adequate updater exists in

the bounded updater grammar, backtrack the feature choice rather than declaring the original query impossible. This addresses the central danger of choosing one appealing auxiliary too early.

In the two-letter example, neither “seen a ” nor “seen b ” alone separates all edges; the pair does, and its componentwise updates are trivial. The script checks this exact feature-cover problem. For a maximum-with-default task, distinguishing empty from nonempty suggests **Option A**; for paired extrema, it suggests an option-wrapped pair. These suggestions still depend on the available comparison operation and on the proof obligations induced by the contract.

What completeness can actually mean

Enumerate all feature subsets within a bound, not just a greedy minimum; enumerate all allowed updates and finishers; retain alternatives when a proof attempt is unresolved. Under finite, complete checking assumptions this is complete relative to that combined space. It is not complete for all list homomorphisms, all catamorphisms, or all useful Lean programs.

For a polynomial functor F and its initial algebra, the same principle is the algebra equation $\phi \circ \text{in} = a \circ F\phi$. Constructor-wise closure obligations replace the single list step. Trees require simultaneous information from all recursive children. An auxiliary that merely fits sample outputs but cannot be updated from the children’s states is not a valid carrier implementation. The algorithm’s most valuable output may be a small, explicit unrealized obligation rather than an ungrounded new type.

4.4 R2.4. Is there a complete, practical motive grammar for primitive recursion with parameters?

Answer. Completeness is available for a deliberately bounded normal-form grammar. A small universally complete and uniformly efficient grammar is not supplied by the fold theory. Separate parameter generalization, result strengthening, and structural recursion instead of mixing all type inventions into one frontier.

Start with three families: the motive directly inferred from the goal; function-valued motives obtained by generalizing a dependency-closed set of existing parameters; and bounded product/option/continuation strengthenings of the result. For a natural-number recursor, these include

$$M(n) = T(n), \quad M(n) = \prod_{s:S(n)} T(n, s), \quad M(n) = T(n) \times U(n).$$

Their well-formedness and universe constraints are part of the search. A generalized parameter may depend on the recursion index, so it must be abstracted with its telescope rather than replaced by a nondependent arrow.

A finite syntax bound gives an enumerable candidate set only after its terminals, literals, universe expressions, and auxiliary type-former choices are fixed. Type-check each motive, deduplicate only with justified equality, and then elaborate its branches. This yields a clear bounded completeness statement. It does not promise that the bound contains the intended accumulator.

A practical ordering is guided by failure information. If the recursive call changes a parameter that the induction hypothesis fixes, generalize that parameter and its dependencies. If a result lacks information required by the next step, try a product with a feature selected by R2.3. If the recursion is a right fold but the desired update threads a left accumulator, try a

function carrier. Natural numbers should not trigger all carrier productions at once; an ordinary primitive-recursive case deserves a cheap direct path before generalized schemes.

This is also a reason not to state that existing dependent search never enumerates useful recursion. Canonical’s examples show that important indexed recursor applications are found when their motive is inferable; supplied generalized principles can expose additional proofs to its search.[16] Leant2’s missing layer is selective invention and organization of such generalizations, not the mere possibility of applying a recursor.

4.5 R2.5. How do induction generalization and lemma discovery transfer?

Answer. Transfer their *failure explanations* and conjecture–test–prove discipline, not the assumption that a conjectured invariant is true. The closest practical bridge is an auxiliary state together with an invariant explaining why it is sufficient.

HipSpec combines theory exploration, testing of conjectured equations, and inductive proof attempts; only proved equations become trusted lemmas.[22] That architecture suggests a two-level Leant2 loop. The program search proposes an auxiliary or worker type. A proof search tries the corresponding strengthened invariant. A failed proof provides information about a missing relation, parameter, or helper, but not a certified reason to discard every program using the current outer scheme.

For example, a reverse worker with accumulator should be paired with an invariant of the shape

$$\text{revAcc}(xs, acc) = \text{reverse}(xs) ++ acc,$$

universally quantified over acc . Fixing $acc = []$ before induction hides the statement needed for the recursive step. The operational repair is a generalized parameter, while the proof-side repair is a stronger induction hypothesis. These are two views of the same missing dependency.

Rippling-style annotations can be approximated initially by a simpler analysis: compare a failed branch’s desired expression with the induction-hypothesis result, record which constructors or arguments prevent matching, and generate a bounded candidate generalization. If the mismatch is an accumulating argument, generalize it; if it is a missing result component, propose a product carrier; if it is an algebraic reassociation, retrieve or synthesize a lemma rather than invent a new state type.

Use a small conjecture budget and preserve a trace from each candidate invariant to the failed obligation that motivated it. Tests quickly reject false equations; a surviving equation remains a conjecture until Lean checks a proof. A theory-exploration bank can be reused across tasks only with its environment and axiom profile recorded. No current, turnkey Lean implementation of the entire HipSpec/rippling loop was established by this review; it is an architectural adaptation, not an available import asserted without evidence.

5 R3: Evaluation of programs with holes

5.1 R3.1. How far do live bidirectional evaluation and higher-order examples extend?

Answer. Further than the proposal suggests, but not all the way to a certified evaluator for dependent Lean terms. Smyth’s distinction between its presentation calculus and its implementation matters: the paper’s core is deliberately restricted, while its implementation supports higher-order examples and removes the trace-completeness requirement.[23] The

appropriate research gap is a dependency-correct adaptation to Lean, not simply the first use of higher-order observations.

A hole closure is not just a hole identifier and an untyped list of values. For Lean, a hole has a local telescope

$$\Delta = (x_1 : A_1, \dots, x_n : A_n(x_1, \dots, x_{n-1}))$$

and a result type $B(\Delta)$. An occurrence carries a well-typed substitution $\sigma : \Gamma \rightarrow \Delta$, so the demanded result has type $B[\sigma]$. Two occurrences of the same hole may inhabit different fibers. Unevaluation must compare or relate their typed substitutions, not demand equality between results of unrelated types.

This is where a naive port breaks first. Evaluating a vector-producing hole can expose an index needed to type a downstream observation. Substituting an untyped angelic vector of the wrong length is not conservative execution; it changes the program language. Likewise, a proof or dictionary in the local environment may determine which reduction is legal. A correct residual representation retains the telescope, substitution, and dependencies even when some of them are computationally irrelevant after closure.

Higher-order demands can initially be represented by finite application observations: a function hole is constrained by the applications actually made by the surrounding program. Those observations may be richer than ordinary input-output pairs, and they need not determine the function extensionally. When an observation cannot be inverted within the supported fragment, return a suspended constraint. Do not guess an arbitrary finite graph for the unknown function and treat it as its entire semantics.

Contract decomposition should also be typed and certified. For each extracted observation $O_i(p)$, obtain a theorem

$$C(p) \longrightarrow O_i(p).$$

That implication is sufficient for refutation. To reconstruct a proof of the original contract from all observations, obtain the reverse direction or an appropriate assembly theorem as well. Splitting a conjunction is straightforward. Interpreting an arbitrary BEq result as propositional equality is not: either keep the Boolean contract literally, or require and use the relevant lawfulness evidence. This is especially important for user-defined instances and dependent data.

5.2 R3.2. What incremental normalizer is suitable for assignment and rollback?

Answer. There is strong reusable methodology for dependency-tracked incremental computation, but this review did not establish a drop-in, verified Lean-hole normalizer with the required rollback semantics. Design the cache around *from-scratch consistency*, immutable versions, and typed dependency identities.

Nominal Adapton’s incremental-computation work makes explicit names and from-scratch consistency central.[25] That is a valuable specification for Leant2’s cache: a reused answer must agree with evaluating the same observation afresh in the restored branch. It is not enough that the cache generally improves speed on editor-style forward updates.

A cache key should include the frozen observation identity, its typed environment substitution, environment fingerprint, transparency and rewrite-rule version, and the versions or values of all assignment cells on which the result depends. Dependencies include expression and universe metavariables, delayed-assignment links, and class-resolution choices. Cache entries may be immutable and shared by branches. Mutable watcher lists must be restored with branch state or indexed by branch lineage.

Proposition 5.1 (Dependency-valid cache reuse). *Suppose an evaluator is deterministic under a fixed semantic configuration and records a sound over-approximation of every mutable cell read in producing a result. If the configuration and all recorded cell values are unchanged in another state, the result can be reused in that state.*

Proof. Replay the evaluation. Every configuration-dependent choice is unchanged. Every read returns the same value, so the same next instruction is taken and the same subsequent reads are made. Induction over the evaluation trace yields the same result. A missing dependency invalidates the argument; an extra dependency merely causes unnecessary recomputation. $\square$

For call-by-need evaluation, record dependencies of the demanded computation, not every hole anywhere in the entire term. An observation suspended on one hole need not rerun when an unrelated proof hole changes. When that hole becomes assigned, resume from a cached residual closure if its captured environment remains valid. On backtracking, the older assignment version naturally reuses its older residual entry.

The inspected implementation already records residual blockers in an `IO.Ref` and uses saved metavariable states for alternatives. It also has special logic for delayed assignments, and its partial-instantiation routine explicitly warns that intermediate expressions are used for reduction rather than assignment.[12] These are concrete places to add branch/version tests. They are not, from inspection alone, a demonstrated false-result bug. The decisive tests assign a hole, decide an observation, restore an ancestor, assign it differently, and compare cached evaluation with fresh evaluation, including dependent local types and delayed assignments.

5.3 R3.3. What is the cheapest route to a fast conservative evaluator?

Answer. First test the proof-producing evaluator already present in the pinned Lean release. Then add a small supported fragment with certificates or a proved semantic connection, not an attempted wholesale replacement of kernel conversion.

Lean 4.34.0's `Lean/Elab/Tactic/Cbv.lean` exposes `cbv` and `decide_cbv`. The latter applies `of_decide_eq_true` and invokes `cbvDecideGoal`; current official documentation describes proof-producing call-by-value evaluation, including equational support useful for recursive definitions.[29, 30] This is a much more concrete starting point than treating every faster evaluator as a fresh trusted compiler. Availability was checked in source; no performance or open-hole completeness claim is made here.

The safest early pipeline is asymmetric. A fast evaluator proposes that an observation is false. Before a hard prune, a certificate checker verifies the corresponding negative observation in the correct context, or a verified fragment evaluator supplies that certificate directly. Unsupported operations, timeouts, and failed proof construction yield *unknown*. They are not refutations. Compiled execution can be valuable for ranking and for finding counterexample candidates, but its output is not automatically a strict-profile proof.

For a closed candidate p and extracted necessary observation $O(p)$, a checked proof of $\neg O(p)$ excludes p . For an open partial program, the required statement is stronger: all permitted completions fail, or no completion satisfying the original contract exists. A proof about one arbitrary filling does not justify dropping the whole branch.

Proposition 5.2 (Conservative fragment refutation). *Let $\llbracket p \rrbracket$ be a partial-program semantics whose set of outcomes contains the outcome of every well-typed permitted completion. If a necessary observation is false for every outcome in that set, the partial program has no contract-satisfying completion.*

Proof. A satisfying completion would produce an outcome in the over-approximation and would satisfy the necessary observation, contradicting the assumed universal failure. $\square$

A constructor/lambda/recursor fragment can implement a more concrete version: reduce only justified beta, constructor, projection, and supported arithmetic steps; leave unknown applications neutral; produce a reduction certificate or appeal to a proved interpreter theorem. Proof-producing rewrites should record the theorem and its side conditions. A residual “stuck” state can still contain useful satisfied observations and exact blocker information.

Lean4Lean is valuable as a specification and metatheory resource, not an automatic extraction button for a complete, fast, verified incremental normalizer. Its published program is explicitly towards verification, with later work extending the mechanized metatheory.[26, 27] Reusing definitions and proof structure is realistic; proving the interaction of holes, dependent substitutions, caches, and rollback remains additional work.

Differential testing against the kernel and the existing evaluator should include closed terms, partial closures, substitution stability, exceptions, proof-irrelevant arguments, and rollback. Such tests find implementation mistakes. They do not establish Proposition 5.2 for all terms, and the article does not replace a missing proof by a large test count.

5.4 R3.4. Can top-down angelic search have termination and completeness guarantees?

Answer. Yes for an explicitly bounded top-down language with fair enumeration and conservative constraints. Burst’s guarantee cannot simply be transferred without rebuilding the argument for Leant2’s representation and checking discipline.

Burst uses angelic execution together with a bottom-up representation and refinement of candidate specifications.[24] Top-down search can instead attach existential result variables to demanded recursive calls while the surrounding syntax remains incomplete. The critical condition is that these variables encode an *over-approximation* of concrete recursive behavior.

Identical calls must share a result constraint, including all explicit and implicit computational parameters. Treating two occurrences of $f(x)$ as unrelated angels may falsely make $f(x) \neq f(x)$ appear satisfiable. This is acceptable only as an intentionally loose over-approximation that is never mistaken for a successful program; sharing the call improves precision. More dangerously, assigning a convenient angelic value and rejecting the branch when that one value fails is an under-approximation and can lose valid programs.

An observed equation $f(v) = w$ may be used as an assumption about that call when it is a consequence of the original contract. The resulting pruning is *contract-relative*: it excludes completions satisfying the contract, not necessarily every arbitrary completion. If the contract permits several outputs at v , retain their relation instead of choosing one output as if it were forced.

Theorem 5.3 (Bounded top-down completeness). *Fix a grammar and bounds giving a finite reachable state graph, including typed skeletons, recursion capabilities, angelic constraints, and proof plans. Suppose no processed alternative is expanded repeatedly, every pending expansion is eventually scheduled, every hard prune excludes only states with no successful completion, and each complete-candidate check terminates with a decision. A search that stops when its frontier is exhausted terminates and finds a successful candidate whenever one exists in the bounded grammar.*

Proof. The finite grammar induces a finite collection of bounded alternatives, after duplicate state identities are handled consistently. Fair scheduling eventually processes every alternative

not pruned. Conservativity ensures that at least one path to any existing successful candidate survives. The complete-candidate decision assumption eventually recognizes that candidate. If there is no successful candidate, every alternative is eventually decided or conservatively pruned, so the search terminates. $\square$

The decision assumption is substantive. If proof search or evaluation can return unknown indefinitely, replace the theorem’s termination/exhaustion conclusion by an honest partial result. Fuel exhaustion is not false. If syntax size is bounded but arbitrary literals or auxiliary declarations are not, the space is not finite. These qualifications are exactly what a useful Leant2 theorem should state instead of claiming completeness merely because all learned proposals are eventually placed in a queue.

6 R4: Indexed families and motives

6.1 R4.1. Is occurrence-based motive enumeration complete and closed under accumulation?

Answer. Enumerating subsets of finitely many eligible occurrences is terminating and complete for that *syntactic template class*. The class is not closed under invention of new accumulator parameters or strengthened result types.

For a goal containing indexed terms, choose a dependency-consistent collection of maximal occurrences, abstract them with fresh variables, and type-check the resulting motive. There are finitely many such collections. The hard work is maintaining a well-formed telescope when later types depend on earlier values; not every syntactic subset determines a legal abstraction. Deduplication should compare well-typed motives under a controlled definitional-equality procedure, with assignments isolated by a saved state.

This can recover the direct vector-map motive

$$M(n, x) = \text{Vec}(B, n)$$

from a goal that transforms $x : \text{Vec}(A, n)$. It cannot manufacture a new result component $U(n)$ or a new accumulator type S that occurs nowhere among the selected occurrences. Thus a claim of completeness for primitive recursion with arbitrary accumulating parameters would be stronger than what the occurrence grammar expresses.

A useful combined grammar has three explicit dimensions: occurrence abstraction, dependency-closed parameter generalization, and bounded carrier/result strengthening. Search them in that order and record which dimension repaired a failed motive. The number of trials then has a declared bound instead of being hidden inside higher-order unification.

For a parameter-generalization choice, close the selected set under type dependencies: if the type of y mentions x , moving x across an induction boundary may require moving y as well. Conversely, reverting all context entries unnecessarily turns every branch into a larger function problem. The dependency graph from R1 provides a principled starting approximation, while failures involving a changed accumulator refine it.

Canonical’s indexed examples show why direct inference should remain the cheap path.[16] The source restriction in current Leant2 is much sharper than a mathematical impossibility: its inspected structural-recursion rule excludes inductive types with indices and excludes **Nat**. [12] A controlled indexed path can therefore be tested before implementing a general motive synthesizer. Its success should be reported separately from success on invented generalizations.

6.2 R4.2. How should search represent transports without losing canonicity?

Answer. Keep transport evidence at the typed boundary, normalize a restricted class of indices, and distinguish proof irrelevance from equality of transported data. A setoid or heterogeneous-equality representation can help organize constraints, but does not remove the need for valid casts in the final term.

There are three separate equalities. Definitional equality allows the kernel to identify types without an explicit proof. Propositional equality supplies a proof that permits transport. Observational equality concerns selected computations and is usually too weak to justify substituting dependent terms. Conflating them makes either the search incomplete or its internal typing invalid.

One small correction illustrates the distinction. For the ordinary recursive definition of natural addition, $n + 0$ reduces to n ; $0 + n$ is the more appropriate generic example of an equality normally handled by a theorem or induction rather than the same direct reduction. Use explicit source-version behavior when designing an index normalizer, not the superficial presence of an addition symbol.[32]

A practical representation stores a canonicalized index expression, a proof from the original index to that expression, and a typed term transported along the proof where needed. Restrict the initial normalizer to terminating oriented rewrites and the supported arithmetic fragment. When a new equality is discovered, record its provenance and orientation. Do not assign zero search cost to unlimited casts: that creates arbitrarily many equal-cost derivations even if the final display hides them.

Lean’s proof irrelevance identifies proofs of the same proposition. It does not identify arbitrary index expressions, and it does not authorize erasing dependencies from a term before checking. Proof-irrelevant components may be ignored by an appropriate ranking feature, while the complete term and its type remain available for certification. Likewise, display simplification may hide routine casts only if the emitted source or replay artifact still re-elaborates to the intended well-typed result.

A typed e-graph or setoid could share terms connected by checked equalities. Its congruence rules must respect dependency, and extraction must reconstruct the transports. Heterogeneous equality is useful when comparing terms whose types differ, but turning such a comparison into a homogeneous goal requires appropriate type-equality evidence. Start with small local equality classes around index obligations rather than announcing one global canonical form for all Lean terms.

6.3 R4.3. What unfolding policy is supported by experience in existing search?

Answer. Both systems provide concrete evidence for a configurable, selective policy rather than unrestricted normalization up front. Canonical describes unfolding reducible definitions, handling instances, and exposing function types hidden behind definitions, while using its own search representation.[16]

CoqHammer’s official `sauto` documentation is particularly explicit. The `unfold:` option names definitions eligible for heuristic unfolding; `unfold!:` requests unconditional unfolding. The default list is empty, with special treatment of logical definitions. Reduction with `simpl` is enabled by `red: on` and is eager by default through `ered: on`; these can be changed. The documentation also separates simplification on context change from backtracking solvers, and states that `sauto` does not itself perform induction.[37]

These are documented policy choices, not comparable timing results. They support exposing reduction strategy as an internal, versioned policy whose effect can be measured, rather than assuming a single transparency level is best for proofs, data synthesis, contracts, type families, and user instances.

The proposed Leant2 policy has four tiers. First expose the outer type constructor and function spine. Next normalize recognized index expressions with a terminating, proof-producing simplifier. Then unfold a definition demanded by a stuck scrutinee or by a failed head comparison. Finally offer a bounded broader-unfolding fallback. Attach the tier and unfolded declarations to cache keys and failure records.

A stuck type family is a useful trigger. If $F(?n)$ cannot expose a head until the index is known, do not repeatedly unfold F without progress. Ask whether a constructor-form split of the index is allowed by the grammar, or whether another goal can determine it. The resulting narrowing step is a search alternative, not a normalization fact. It must share the same metavariable assignment state as the rest of the query.

The desired experiment compares fixed transparency, demand-driven unfolding, and pre-normalization on separately labeled workloads. Record time spent in reduction, failed unification, and repeated normalization; count solved and certified tasks; and preserve equivalent environments. A translation to another assistant changes elaboration and libraries, so cross-system results should be informative case studies, not a raw speed contest presented as an implementation theorem.

For a reproducible cross-system experiment, pin the actual tool revisions and options rather than relying only on current web documentation. Record which definitions are explicitly eligible for unfolding and whether context-change simplification is eager. The documented `sauto` options give a concrete comparison matrix; their best setting for Leant2's workloads remains unmeasured.

6.4 R4.4. Can Lean's motive computation be called directly and diagnosed?

Answer. Yes. In the pinned release, public meta-level functions already expose important pieces of the computation. They provide useful failures, but they do not by themselves search over all generalized motives.

The inspected `Lean/Meta/Tactic/Induction.lean` contains `getMajorTypeIndices`, `mkRecursorAppPrefix`, and `Lean.MVarId.induction`. The prefix function abstracts the major premise and indices from the current target, chooses recursor universe levels, and checks restrictions such as elimination from `Prop`. The induction function reverts indices and the major premise with their dependencies, reintroduces them, and builds branch goals.[28]

The source also shows limitations that a caller should expose rather than conceal. The index-extraction helper expects suitable free-variable indices, rejects repeated or improperly dependent indices, and reports malformed recursor or major-premise situations. A composite index such as $n + 1$ may require preliminary generalization or a higher-level elaboration path. It is not evidence that no motive exists.

Wrap these services in a transactional trial interface. A trial receives a prepared local context and a proposed abstraction/generalization policy; it either returns branch obligations and a replayable transformation, or a structured failure category. Useful categories include unsupported index form, universe/elimination restriction, ill-typed motive, unresolved instance, resource limit, and ordinary branch failure. Keep exception text as diagnostic detail, but do not train a refuter on an undifferentiated Boolean failure.

Cost bounds should apply to each trial. Restoring only a goal list is insufficient if the trial assigned universes, created delayed constraints, or changed elaborator state. Start by reusing the same saved-state discipline as the existing search, then test that repeated failed motive trials leave the next trial observationally unchanged.

This answers the API question more concretely than a new general-purpose motive-inference service would. A thin, version-pinned adapter around existing functions is a plausible first implementation. Whether it is cheap enough on the intended corpus is unmeasured here and belongs in the indexed-induction milestone.

7 R5: Recursion from incomplete examples

7.1 R5.1. Does a capability interpreter agree definitionally with the realized program?

Answer. Not in general. Design for a propositional simulation theorem, and exploit definitional reduction only in cases where it actually holds. Structural recursors, generated equations, and well-founded recursion have different reduction behavior.

A recursive-call capability can have the dependent type

$$\text{rec} : (y : X) \rightarrow R(y, x) \rightarrow B(y),$$

where a fixed proof establishes that R is well founded. The body language may invoke `rec` only with such evidence. Structural subterm paths can generate some evidence cheaply; a measure-decrease obligation may require arithmetic reasoning. The capability is a typed permission, not an unchecked annotation that a call “looks smaller.”

The realized Lean definition may use a structural recursor, course-of-values recursion, or well-founded recursion selected by the definition elaborator. Official documentation explicitly distinguishes cases where recursion equations reduce directly from cases where equational theorems are required.[31] An arbitrary search interpreter is not thereby definitionally equal to every generated term. A raw `WellFounded.fix` expression also needs care when executable code, rather than only a kernel term, is required.

A suitable theorem has the following shape:

$$\text{RunFuel}(k, b, x) = \text{Done}(v) \implies \text{realize}(b)(x) = v. \quad (5)$$

Its assumptions include well-typedness of b , validity of its recursive capabilities, agreement of primitive operations, and the realizer’s defining equations. A proof proceeds over the interpreter derivation, applying the corresponding realization equation and induction hypotheses for smaller recursive calls. Dependent results require the same typed substitutions and transports on both sides.

The implication is deliberately one-sided. Running out of fuel gives no equation and no refutation. Conversely, termination of a Lean definition does not supply a small uniform fuel bound suitable for every search observation. A total evaluator can eventually finish on a fixed closed input, yet still miss the interactive budget.

For the first implementation, enumerate bodies against typed capabilities, test promising closed bodies in a direct interpreter, and realize only selected candidates through the definition elaborator. Then check the actual contract using kernel-checkable equations or proof-producing evaluation. For hard pruning based on the interpreter, either prove (5) for the fragment or

reconstruct each decisive negative observation. Reusing the pinned *cbv*/equational infrastructure is preferable to assuming that compiled execution has the strict profile’s trust guarantees.[29, 30]

The user-visible result should also distinguish a certified mathematical function from an executable artifact with a verified compilation path. Both may be useful, but one is not a substitute for the other.

7.2 R5.2. What does top-down angelic recursion gain and lose?

Answer. It composes with type-directed search when angels are typed constraints attached to calls. It loses some of the compact sharing and refinement structure supplied by a bottom-up version-space representation, and must recover that sharing explicitly.

A top-down partial body exposes holes with local environments, recursive-call capabilities, and expected result types. Executing an observed input can accumulate equations about the results of demanded smaller calls even before every syntactic hole is filled. This can prioritize a hole whose answer would resolve many observations. It need not fabricate trace-complete examples in advance; live constraints can arise from the current sketch. Smyth and Burst supply different precedents for this interaction between execution and synthesis.[23, 24]

The main representation problem is repeated work. Many top-down sketches contain equivalent subexpressions or induce the same demanded recursive calls. A bottom-up automaton or version space may share them naturally; a naive top-down tree may repeat their constraints and evaluation. Hash-cons typed call descriptors, cache residual closures by branch-valid dependency keys, and share constraint sets without merging incompatible assignments.

Keep three levels of evidence distinct. A forced answer from the original contract is a justified call constraint. An existential answer that would make the current observation pass is an angelic possibility. An actual value obtained by evaluating the closed realized candidate is a concrete result. The first may constrain all accepted completions; the second only guides or participates in conservative existential reasoning; the third supports final checking.

Refinement should block a failed *combination* of skeleton and call assumptions, or add a justified relation, rather than indiscriminately banning the whole skeleton. For example, a failed assumption about one unobserved recursive call does not show that a different result assignment could not complete that body. Nogoods should carry exactly the assumptions on which they depend.

Relative completeness is the bounded theorem in R3.4, with the recursion scheme included in the grammar. It is not a new unqualified completeness result for top-down synthesis of arbitrary general recursion. The practical evaluation should compare full trace examples, sparse examples with neutral calls, and sparse examples with angelic constraints under the same body and scheme bounds.

7.3 R5.3. Is there a canonical small basis of recursion schemes with measured coverage?

Answer. No particular minimal basis with the requested Leant2 coverage was established by the available evidence. There is a defensible small *engineering portfolio*, and its coverage can be measured without claiming canonicity.

A useful initial portfolio is: structural recursion on one input while generalizing the others; indexed structural recursion; product-strengthened or mutually organized workers; and well-founded recursion under a bounded grammar of natural-valued or lexicographic measures. These

are expression mechanisms, not independent mathematical classes: one scheme can encode another at a substantial search or runtime cost.

Several apparent needs for new schemes can instead be handled by parameters or state. A zip-like function can recurse structurally on one list while inspecting the other. Fibonacci can be computed by primitive recursion with a pair of consecutive values. An efficient tree flattening worker can carry an accumulator or continuation. A merge-like function with branches decreasing different lists may be expressed by nested structural organization or a measure such as combined length; the cheapest representation depends on the allowed language. These examples motivate the portfolio but do not measure its coverage.

The basis should be ordered by *total search cost*: choosing a scheme, finding its body, proving decrease, realizing it, and certifying the contract. A tiny measure grammar that opens difficult proof obligations may lose to a larger structural body. Admit only measures whose decrease proofs lie in a declared proof-plan fragment before attempting broad measure invention.

Report coverage on a corpus stratified by recursion structure, not only by function name. For each task record whether the target is expressible in each scheme, whether a supplied scheme enables synthesis, and whether the scheme is successfully invented. Separate body search from scheme selection. Otherwise a failed synthesis of an easy structural function can be misreported as evidence that the scheme portfolio lacks expressive power.

Mutual and nested inductives deserve a dedicated later track because elaborator-generated recursors and helper declarations affect both representation and cost. Coinductive or intentionally partial programs require a different semantic and acceptance profile; they should not quietly enter a total Lean synthesis claim through an interpreter fuel mechanism.

8 R6: Contracts beyond decidable observations

8.1 R6.1. How should value holes and proof holes be interleaved?

Answer. Use dependency-aware obligations with staged proof effort. Perform cheap logical propagation early, postpone expensive unstable obligations, and synthesize local value–proof pairs when the property follows the recursion. There is no general analogue of decidable refinement checking for arbitrary Lean propositions.

Synquid is a useful reference because it prunes incomplete programs using polymorphic refinement types and a restricted logical checking discipline.[33] The restriction is part of the achievement: replacing that logic with all Lean propositions does not preserve a decidability or cost guarantee. Leant2 can nevertheless borrow the separation between a program sketch and the necessary local obligations it induces.

Maintain edges from each proof obligation to the value and type metavariables on which it depends. Run inexpensive normalization, reflexivity, constructor reasoning, and obvious arithmetic constraints as soon as they are informative. A proof tactic that assigns data metavariables has made a program-search choice; isolate that choice as a branch rather than silently committing it across alternatives. More expensive tactics should wait until their relevant dependencies are stable, not until every unrelated hole in the entire program is closed.

For recursive properties, lift the result into a dependent pair. A length-preserving transformation can target

$$(xs : \text{List } A) \rightarrow \{ys : \text{List } B \mid \text{length}(ys) = \text{length}(xs)\}.$$

In a constructor branch the induction hypothesis supplies both a transformed tail and its length proof. The value part is synthesized first enough to expose the local equation; the proof part

often closes by rewriting and arithmetic. This is considerably better structured than asking a general tactic to understand a large completed recursor term afterward.

A concrete implementation obstacle is visible in the current source: `proofPortfolio` returns immediately when the target contains free variables, as well as when it contains metavariables. Its tactic sequence is `rfl`, `decide`, `simp`, and `omega`.^[12] A local induction-hypothesis proof goal normally contains legitimate free variables from its context. The necessary repair is not merely adding another tactic to this closed-goal portfolio; it is supporting contextual obligations with correct abstraction and branch isolation.

Testing remains a refuter for universal properties. A generated counterexample must be a well-typed input, including its index and evidence for dependent domains, and its failure must be confirmed in the accepted semantics. Reuse checked counterexamples across candidates. Passing them only increases confidence or priority; it never replaces the universal proof.

8.2 R6.2. Can the easy induction–simplification–arithmetic class be recognized?

Answer. A useful *sufficient proof-plan class* can be recognized and certified. There is no reason to expect a complete syntactic recognizer for every theorem that a resource-bounded mixture of induction, rewriting, and arithmetic might eventually prove.

Define a finite grammar of induction plans. A plan specifies a recursion argument or generated functional induction theorem, a bounded parameter generalization, permitted unfolding equations, a terminating rewrite set, and a leaf logic such as quantifier-free Presburger arithmetic with already established algebraic facts. The recognizer does not guess that the goal “looks easy”; it checks that one of these plans produces leaves in the admitted fragment and that the leaf solver constructs proofs.

For example, a plan for length preservation unfolds the program once per constructor, rewrites the induction hypothesis’s length equation, and reduces the successor arithmetic. A plan for sortedness may additionally require a comparison law and a membership invariant. A permutation property typically needs a suitable list algebra lemma, not just arithmetic. The presence of a recursively defined program does not make all its properties part of the same class.

Proposition 8.1 (Certification by a local induction plan). *Suppose a generated induction principle reduces $\forall x, P(x)$ to finitely many branch obligations. If each branch is transformed by checked equalities into a proposition for which the selected leaf procedure returns a kernel-checkable proof, then the plan yields a kernel-checkable proof of $\forall x, P(x)$.*

Proof. Compose the equality transports and leaf proofs to obtain each branch proof, then apply the induction principle. Every composition step is represented in the resulting proof term and is independently checked by the kernel. $\square$

Membership in the successful bounded-plan class is decidable when plan enumeration and leaf checking terminate. Failure of the recognizer means “outside the successful plans explored,” not “unprovable.” An expensive prover can then be scheduled as a separate lane. This gives a rigorous version of the proposal’s wish to avoid wasting the whole budget on unsuitable contracts.

The resulting plan receipts are also useful training data: they identify the generalization, equation lemmas, and leaf-theory boundary that actually sufficed. They are more informative than a binary label saying that a monolithic tactic succeeded within a particular machine’s time limit.

8.3 R6.3. Which automated-induction facilities are usable now?

Answer. Use existing induction and automation APIs for bounded local plans; regard broad lemma invention and rippling as additional research. The current Lean reference documents `fun_induction`, which uses a function’s associated induction principle, including supported structural and well-founded definitions. It also documents the familiar simplification, arithmetic, and `grind` machinery.[30] Availability and exact behavior must be pinned when integrating them; a current reference is not a substitute for a release-specific test.

The first practical lane should construct a functional or structural induction plan, expose the recursive equations, and run small leaf tactics in the branch contexts. For a generated candidate, temporary checked declarations may be needed to obtain stable equation and induction theorem names. Keep these declarations in a scratch environment and include their dependencies in replay. A synthesized recursor expression is not automatically accompanied by every named theorem that a source-level definition elaborator would generate.

Aesop can serve as a configurable local proof search; Canonical can test suitable dependent inhabitation problems; and LeanHammer provides a current example of premise selection combined with symbolic search and proof reconstruction.[15, 16, 36] None is evidence that arbitrary accumulator invariants or missing induction lemmas will be invented under Leant2’s default budget. Premise-selection gains on theorem corpora should not be presented as gains on program synthesis with examples.

HipSpec’s proven-lemma bank and conjecture-testing loop remain a useful adaptation target, as explained in R2.5.[22] The conservative first version is a small bounded grammar of equations suggested by failed branch obligations, with generated tests followed by Lean proof attempts. An unproved equation must never become a simplification rule used to certify later candidates.

Every tactic adapter needs a resource contract: heartbeat or logical-work limit, wall-clock cancellation policy, restoration after failure, and a proof term checked under the query’s axiom profile. Remote premise selection or model-assisted proof proposals should be optional and outside the core-only path. Network latency, process startup, and proof reconstruction must all count toward end-to-end measurements, not be hidden as preprocessing.

8.4 R6.4. What scheduling policy should pay proof debt?

Answer. Begin with a transparent cost-sensitive priority and a fair reserve, not a claim of optimality from a generic bandit theorem. Proof attempts are correlated, stateful, and sometimes reusable; their payoff depends on the remaining synthesis frontier.

For an obligation o and next proof quantum q , a useful heuristic is

$$\text{priority}(o, q) = \frac{\widehat{\text{Pr}}(\text{certification in } q \mid \text{features}(o)) \cdot \text{value}(o)}{\widehat{\text{cost}}(o, q) + \text{switchCost}(o, q)}.$$

Here value reflects the candidate’s quality and whether proving the obligation unlocks many branches. This is a proposed scheduling score, not an admissible bound and not an optimality theorem. Estimates should be calibrated on held-out traces and frozen within replayable policy epochs.

Use a deterministic tier ladder: cheap local proofs first, then richer local plans, then global or retrieved-premise search. Give a fixed share of logical work to older unresolved obligations so an attractive stream of new candidates cannot starve them. Resume a proof state when its dependencies remain valid; otherwise restart with the reason recorded. A newly found

counterexample can invalidate many proof debts at once and should be tested against pending candidates early.

The scheduler should optimize time to the *first certified useful answer*, not time to the first syntactically complete or test-passing program. Measure the gap between these events, proof-attempt success by tier, work lost to changing dependencies, and switch/reconstruction overhead. A policy that finds many unproved programs quickly may be worse for Leant2 than a slower-looking policy that certifies one promptly.

A useful baseline is intentionally simple: round-robin proof quanta among the best few stable candidates, with a small reserved synthesis quantum and cached counterexamples. More elaborate bandit or anytime policies should beat that baseline on the same end-to-end budget before becoming the default. No optimal scheduling theorem for this full dependent, dynamically changing setting is claimed here.

9 R7: Component retrieval at library scale

9.1 R7.1. What abstraction is useful and sound for dependent signatures?

Answer. Use an AND-style typed hypergraph with refinable shape information, preserve variable correlations and necessary universe/class constraints, and map genuinely unknown dependent heads to an over-approximation. A simple graph of type heads is a useful ranking index, not automatically a sound completeness-preserving filter.

A component with type

$$(x_1 : T_1) \rightarrow \cdots \rightarrow (x_n : T_n(x_1, \dots, x_{n-1})) \rightarrow U(\vec{x})$$

requires its arguments jointly. An ordinary edge from each argument head to the result head can incorrectly suggest that any one argument suffices. Model it as a hyperedge with compatible substitutions. Nonlinear use of Lean locals also matters: a term can normally be reused, so a consuming Petri-net interpretation must explicitly support contraction or use a nonconsuming reachability abstraction.

The initial abstraction can retain a bounded function spine, outer constructors, repeated type-variable identities, simple index patterns, and conservative universe relations. For example, the two occurrences of A in $A \rightarrow \text{List } A$ should not become unrelated wildcards. A function-valued argument should not be confused with a value of its codomain. Preserve dependencies that are cheap enough to rule out common spurious compositions.

Dependent type families require particular care. If $F(b)$ reduces to Nat for one Boolean and Bool for another, an unresolved $F(?b)$ cannot be permanently indexed under only one of those heads. Use a union or top shape until constraints refine it. Likewise, collapsing universe-polymorphic signatures to a Prop instance is not a sound abstraction for constructing a value at an arbitrary required universe.

State the abstract simulation invariant: every concrete application allowed by the search must have a corresponding abstract hyperedge instance. It is then safe to eliminate components proved unreachable *within the declared composition bound*. Choosing only paths of length three is a deliberate language restriction or heuristic; it is not a proof that a length-four solution is absent.

On a spurious composition, refine the abstraction with the distinguishing type/index correlation and retry. Merely failing concrete unification on a path is not yet type-guided abstraction refinement unless the abstraction changes. Do not refine from a timeout or from an unresolved

instance whose type may later become known. Keep a conservative wildcard fallback lane when retrieval remains heuristic rather than a proved complete filter.

9.2 R7.2. Has abstraction refinement already been combined with type classes?

Answer. Yes. Hoogle+ explicitly treats type classes through dictionary passing, with instance-producing components and additional handling for functional dependencies.[34] This directly answers the question’s historical premise. Its reported evaluation uses 291 components and 44 queries; it should not be summarized as an established dependent-language solution over tens of thousands of components.

In the dictionary view, a class constraint becomes an explicit argument of a dictionary type. An instance declaration becomes a component that constructs such a dictionary from its own prerequisites. Method use then becomes an ordinary typed application. This is exactly the kind of prerequisite that belongs on an AND hyperedge. Hoogle+’s stated restrictions, including its treatment of higher-kinded type variables, matter when comparing it with Lean rather than assuming the entire setting transfers unchanged.[34]

Lean adds dependent dictionaries, universe polymorphism, local instances, priorities, and elaborator-specific search behavior. A retrieval abstraction can approximate the availability of a dictionary, but final resolution must use the actual local context and be checked as part of the term. A class precondition indexed only by a pretty-printed type name will be too coarse for both correctness and performance.

Instance memoization should therefore include the normalized class application, relevant local instance environment, transparency configuration, and assignment dependencies. A positive closed dictionary can often be abstracted and reused. Failure on an open type, such as an unresolved `Add ?A`, is not a permanent negative result for every future instantiation of `?A`. Cache it as suspended with blockers, or scope it to the exact state version.

The recommended prototype is deliberately small: convert selected class-generic providers into explicit dictionary-aware hyperedges, compare abstract reachability with concrete Lean application, and count spurious paths by cause. Only then add larger libraries and persistence. An environment fingerprint should distinguish imported declarations, local additions, and relevant instance changes; an index that survives an incompatible environment update is not merely stale ranking data if it is also used for hard filtering.

9.3 R7.3. Can premise selection serve data goals and example-based retrieval?

Answer. The available infrastructure can inspire a data-goal retrieval adapter, but a theorem-premise selector’s interface and benchmark do not establish effectiveness on program components selected by examples. Build a distinct task representation and corpus, then measure transfer.

LeanDojo provides accessible-premise data and retrieval infrastructure for theorem proving, while LeanHammer combines contextual premise selection with proof search and reconstruction.[35, 36] These are valuable for proposition-typed holes and for proof obligations attached to programs. A data goal such as `List A → List A` may, however, share its type with reverse, filter-like operations, duplication, and identity; examples carry information that a proposition-only retriever was not trained to use.

Define a retrieval request containing the typed local telescope, target, available instances and operations, supported observations, and the allowed profile. The response is a bounded ranked

list of declarations or typed compositions, with provenance. An examples-aware reranker can compare component behavior on closed probes, but a component that fails to solve the task alone may be essential inside a composition. Behavioral mismatch is therefore a ranking signal unless a compositional impossibility argument is available.

Definitions in Std, Mathlib, and other Lean projects offer a potential training corpus: hide the body, retain a permissible specification, and predict components used by an acceptable implementation. This requires more than deleting one definition body. Remove the target declaration itself, generated equation lemmas, compiler-generated helpers revealing its implementation, and closely associated proofs that trivially recover it. Split by definition family and project where possible; a random declaration split can leak almost identical algorithms into training and test.

Generate behavioral observations only through a declared executable and testable fragment. Record which contracts are supplied by humans, extracted from theorems, or generated from a reference implementation. Evaluation must not quietly expose the reference body to the retriever. Use symbol-overlap and type-only retrieval as baselines before attributing improvement to learned behavior features.

Finally, report cold import/index time, warm retrieval time, number of returned candidates, concrete applicability, and end-to-end certified synthesis. A fast ranker that returns too many spurious providers may increase total search time. No numerical data-goal retrieval accuracy or library-scale Leant2 speedup is asserted by this report.

10 R8: Ranking and candidate equivalence

10.1 R8.1. What objective makes ranking defensible?

Answer. Minimum description length under a declared context-sensitive grammar is defensible as a reproducible preference model. It is not a theorem of user intention. Combine it with explicit task policy and evaluate the preference empirically.

For a frozen grammar prior $P_G(p \mid \Gamma, T)$, a simple objective is

$$\text{score}(p) = -\log P_G(p \mid \Gamma, T) + \lambda_1 \text{runtimeProxy}(p) + \lambda_2 \text{dependencyPenalty}(p),$$

subject to the exact certification conditions. The prior can favor idiomatic library components, shallow structural recursion, or small strengthened states. Its choices should be versioned. Grammar-guided synthesis such as Probe gives a precedent for learned enumeration probabilities, not a proof that this score matches every user’s preference.[17]

The current ranking counts unused outer binders, eliminator occurrences, and raw term size, then removes candidates with identical printed forms.[13] Those are useful heuristics but mix different notions of simplicity. A variable mentioned only in an erased proof or a type can count as syntactically used without influencing the data result. Conversely, ignoring an input can be exactly what a valid specification requires. Information-flow penalties should remain defeasible preferences, not a correctness filter.

A better initial score distinguishes explicit computational inputs, implicit type parameters, dictionaries, and proofs. Count the readable program skeleton separately from its certification term. Favor a named library operation when its dependency is allowed and its meaning matches the contract, but do not treat arbitrary large opaque constants as zero-cost shortcuts. State whether the objective measures source readability, derivation complexity, or runtime, because no one number automatically measures all three.

An optimal stopping rule requires the same objective on completed candidates and a valid frontier lower bound. If all unexpanded states have lower bound exceeding the best candidate’s

score, optimality follows for the admitted space. A fixed grace period, or an A^* bound for unrelated production costs, does not prove that statement. Learned priorities can still improve discovery while leaving the final ranking fixed.

Evaluate ranking through blinded pairwise choices, accepted-result rank, edit distance to the ultimately retained program, and time to a satisfactory result. Include deliberately underdetermined contracts and cases where unused inputs are appropriate. A reference implementation is one acceptable answer, not always the only preferred one; exact syntactic rank of that reference should be supplemented by semantic or task-level judgments.

10.2 R8.2. Which equality should group dependent candidates?

Answer. Use several explicitly labeled levels. Definitional equality and checked equality proofs can support strong deduplication; agreement on generated probes supports presentation and heuristic sharing, not unrestricted substitution in search.

The levels are: alpha-equivalent typed syntax; definitional equality under a pinned environment; propositional equality with a checked proof; and equality on a specified finite observation set. Their costs and guarantees differ. Printed-form equality is not a satisfactory foundational identity because pretty-printing can omit parameters, notation choices, or type information. The current printed deduplication is best understood as output cleanup.[13]

For a simple counterexample, $f(n) = 0$ and $g(n) = \text{if } n = 0 \text{ then } 0 \text{ else } 1$ agree on the sole probe 0. A context applying them to 1 separates them. This is checked by the script. In a restricted first-order DSL, an observation relation may be a congruence for the permitted contexts and inputs, enabling sound quotienting. That congruence must be established; it is not inherited automatically by higher-order or dependent Lean contexts.

Dependent outputs add a typing issue before the equality issue. Compare terms in the same fiber after justified normalization or transport, or use a carefully typed heterogeneous relation with the required extraction evidence. Proof fields can often be ignored under proof irrelevance while comparing the underlying data, but a type index carried by the data cannot be erased just because some accompanying equality proof is irrelevant.

The practical policy is to retain candidates and provenance behind an observational bucket, show one preferred representative, and split the bucket when a new distinguishing observation arrives. For hard search pruning, discard a representative only under a relation proved congruent for all future allowed uses, or with an explicit equality certificate. For output diversity, label buckets as “same on these probes” and display a discovered distinguishing input between separate buckets. Do not describe them as mathematically identical without evidence.

For type-former-valued answers, ordinary ground probes may be unavailable or misleading. Group by typed shape and available elimination behavior, with conservative labels. The proposal’s problem of twelve variations of one unhelpful idea is a presentation/ranking problem before it is an extensional-equality decision problem.

11 R9: Learned guidance under a kernel gate

11.1 R9.1. How accurate are models at proposing carriers or helpers from types and examples?

Answer. No verified, directly matching accuracy figure was located. Published successes in premise selection or whole-proof generation do not answer this narrower question. The useful

answer is a reproducible experiment that isolates the proposal’s actual contribution rather than importing an unrelated percentage.

Use a task matrix crossing carrier-given versus carrier-invented and trace-complete versus trace-incomplete observations. Add a separate helper-invention track. For each task, request only a carrier, generalized telescope, helper signature, or structured sketch—not an unrestricted replacement program whose larger contribution would confound the measurement. Compare an enumerative grammar, a hand-designed feature proposer, a frozen small learned model, and an optional larger model under the same downstream search.

Report distinct events: the proposal parses; it is well typed; it is admitted by the stated grammar or extension policy; it permits the reference behavior in a supplied-body test; the body can be synthesized; and the result is fully certified. These rates answer different questions. A carrier can be mathematically adequate but too difficult for the downstream search, while an attractive helper signature can be uninhabited or incompatible with the contract.

Measure top- k proposal coverage and time to a certified answer, including model initialization, token generation, retrieval, parsing, elaboration, and failed proposal attempts. Give the baseline an equal end-to-end budget. Distinguish cold-start local inference, a resident local model, and remote inference; averaging them into one latency number would obscure the interactive tradeoff.

Hold out algorithm families and structurally related variants. Rename entities and vary element operations to test reliance on familiar textbook names, but do not pretend that renaming alone eliminates training contamination. Include underspecified tasks where several carriers are valid and evaluate adequacy rather than exact textual agreement with one expected type. A useful failure report separates missing information in the examples from type errors, representation limits, body-search failures, and proof failures.

This study can answer whether a model’s carrier proposal is worth its cost for Leant2. The present report did not run it. The verified literature on Lean retrieval establishes adjacent capability, not the requested rate.[36, 35]

11.2 R9.2. What makes learned contributions deterministic and auditable?

Answer. Freeze and record the *entire proposal and decision boundary*, not only the winning term. Deterministic proof checking is easier to guarantee than identical model-generated search trajectories.

Temperature zero and a random seed do not, by themselves, specify every numerical implementation choice. A reproducible learned lane should record model/checkpoint identity, tokenizer, prompt, retrieved context, decoding parameters, relevant runtime/version information, the complete ordered proposal batch, and its normalized typed representation. Prefer integer or otherwise explicitly specified scores for the final scheduler. A replay should not have to contact the model again.

Record the grammar and bounds, axiom policy, environment fingerprint, logical work schedule, deterministic tie-breaking identities, and every policy update. For auditing completeness or pruning, also record hard-prune reasons and their scope. Recording only the final accepted proof can reproduce acceptance, but cannot demonstrate that a search fairly covered the advertised language or did not discard a better answer.

```

receipt:
  frozen query, actual target universes, axiom profile
  environment + toolchain + grammar identities
  all proposal batches, in order, with provenance
  logical scheduling events and policy epochs
  decisive observations / prune certificates and scopes
  completed coverage bounds; unresolved obligations
  accepted program and exact contract proof

```

The model interface should return a restricted typed proposal language or raw expression syntax whose interpretation is controlled by Leant2. It should not execute arbitrary generated Lean commands, macros, elaborator code, or IO. The kernel gate constrains accepted logical declarations; it is not an operating-system sandbox for code executed while constructing those declarations. Whitelist permitted proposal forms, resolve names against the frozen environment, and audit generated dependencies.

A core-only default can use the enumerative and feature-based proposers behind the same interface. An optional learned lane can add a proposal ordering without changing the trust profile. If it introduces candidates outside the original grammar, record a new grammar extension rather than claiming it only reordered the original set. If it merely reorders, preserve a fair base lane and the distinction between finite exhaustive completeness and deadline-limited discovery.

There is a useful precedent: CoqHammer’s documentation recommends replacing a successful hammer call with its reconstruction tactic, distinguishing a potentially nonreproducible search from the reusable proof artifact.[37]

Finally, replay should independently recheck the program and contract proof in a fresh environment. That is the strongest and simplest certificate for the result actually used in a correctness argument. The search receipt provides additional accountability about how the result was found; it does not need to make statistical generation part of the logical foundation.

12 Integration plan and acceptance gates

12.1 One architecture, not nine independent add-ons

The nine themes share three central structures: typed dependency information, a bounded space of proposed programs and generalizations, and evidence-bearing observations. Implementing them separately risks nine incompatible caches and nine meanings of “failure.” The integration should instead expose a small set of reusable services.

A *query service* freezes the target, contract, environment, profile, and actual universe interpretation. An *obligation service* tracks value, type, instance, decrease, and proof obligations in one dependency graph. A *proposal service* enumerates ordinary rules, carriers, motives, components, and optional model suggestions. An *observation service* reduces, suspends, or certifiably rejects under a specified semantics. A *scheduler* selects logical events without changing their evidentiary meaning. The *gate* certifies complete results independently of all priority choices.

The current source has natural entry points for this refactoring: `alternative` and `search` for branch discipline, `evalResidual` and `partialRefute` for observations, `structuralRecursion` for schemes, `proofPortfolio` for local evidence, `enumerate` and `rankCandidates` for scheduling/output, and `gate` for the final boundary.[12, 13, 11] These names identify inspected code, not a proposed patch already applied to the repository.

Stage	Change	Gate before proceeding
P0	Freeze semantics and result labels.	Every accepted result carries the actual target and contract; timeout cannot be mistaken for exhaustion; universe specialization is explicit; existing gate checks are preserved.
P1	First-class branch states, dependency logging, and per-observation caches.	Depth-first scheduling reproduces the baseline before policy changes. Assignment/rollback tests agree with fresh evaluation. Unknown instance failures are not cached as global impossibility.
P2	Contextual proof obligations and existing Lean APIs.	Local induction hypotheses can be used safely. Pinned motive helpers and <code>cbv</code> adapters are tested on supplied sketches and adversarial open-hole cases.
P3	Controlled indexed and parameterized recursion.	Vector-map/append-style tasks separate direct motives from invented generalizations; decrease obligations are typed; realization and contract checking succeed independently.
P4	Carrier proposals from features, with closure refinement.	The finite-state examples and reverse counterexample pass. Sample separation alone cannot certify an updater; alternatives survive failed proof attempts.
P5	Typed library retrieval and local proof plans.	Every hard abstract filter satisfies an over-approximation test; class and dependent-head cases are covered; cold/warm costs and end-to-end proof costs are reported separately.
P6	Learned priorities, diversity, and receipts.	Deterministic replay works without model access. All proposals and decisive pruning events are recorded. The learned lane beats fixed-policy baselines without changing acceptance or exhaustion claims.

The order is a dependency order, not a time estimate. Small versions of P4 or P5 can be experimented with earlier, but their hard-pruning features should not become the default until P0–P2 establish the evidence boundary.

12.2 A concrete end-to-end search loop

The following pseudocode states responsibilities rather than promising a particular Lean API signature.

```
freeze query, environment, profile, grammar, and policy
compile necessary observations with implication certificates
initialize persistent root state and deterministic base frontier
initialize optional heuristic frontier and proposal batches

while logical work remains and cancellation has not occurred:
  choose next event under the recorded scheduling policy
  restore its branch snapshot and validate dependency versions
  propagate cheap typing, instance, and observation constraints
  if a checked necessary observation excludes the branch:
    record the scoped refutation and continue
  if obligations are complete:
    realize recursive bodies and reconstruct the exact contract
    run the acceptance gate; retain only certified results
    update output buckets without losing unjustified alternatives
  else:
    select an obligation / dependency component
    enumerate bounded proposals; transactionally elaborate each
    enqueue surviving alternatives in base and heuristic frontiers
    checkpoint coverage, unknowns, policy epochs, and evidence

return certified results plus exact coverage / interruption status
```

A queue entry must be more than a goal and a scalar score. It needs a branch identity, saved logical state, relevant environment version, grammar bounds, accumulated fixed cost, and pending evidence. A compact trail or persistent map may optimize its representation, but the semantic contents should be specified before optimizing them away.

The highest-risk optimization is one that moves information from a heuristic channel into a hard filter without a new proof obligation. Examples are treating a sampled carrier image as exhaustive, treating model nonselection as irrelevance, treating an unresolved instance as impossible, and treating one failed angel assignment as failure of the whole partial program. These transitions should be visible in code review and covered by negative tests.

12.3 What would justify a negative or optimality claim?

For a certified impossibility result, return a checked theorem excluding the target or the contract. For bounded exhaustion, require that every admitted production instance has been explored, merged by an established congruence, or eliminated by a conservative refutation; unresolved checks must be absent. A machine-readable coverage manifest should enumerate bound parameters rather than describe the search merely as “depth 10.” Carrier, recursion, literal, transport, proof-plan, and component-composition bounds are separate dimensions.

For ranking optimality, require a fixed candidate objective and a lower bound for every remaining eligible state, including states delayed in other lanes. A best-first queue does not prove optimality if another lane has an unaccounted cheaper candidate or if the final rank is computed differently. Without these certificates, return “best found,” which is both useful and accurate.

13 Evaluation protocol and results actually obtained

13.1 Separate expressibility, discovery, and certification

Use three nested experimental tracks. In the *supplied structure* track, give the carrier, recursion scheme, motive generalization, and needed components; measure body and proof search. In the *invention* track, remove one supplied structure at a time and measure its discovery. In the *end-to-end* track, give only the advertised query and allowed library. This avoids attributing a type-language limitation to the scheduler or attributing proof failure to carrier prediction.

Stratify tasks into ordinary first-order data construction, Church-encoded folds, finite-state accumulators, higher-order continuations, indexed families, measure-based recursion, and universal properties. Add sparse versus trace-complete observations, small versus broad libraries, and local-instance variations. Preserve both the original repository regression suites and independent held-out families; success on a suite used to tune the heuristic is not evidence of generalization.

Record first test-passing program, first certified program, best found score, number of kernel checks, observation evaluations and cache hits, metavariable/instance work, proof quanta, peak frontier size, and cancellation status. Report end-to-end wall time and deterministic logical work side by side. Do not merge warm-index results with cold process and import costs. For model-assisted runs, include generation and failed proposals in the budget.

The principal ablations should isolate explicit state representation, dependency caching, contextual proofs, carrier proposal strategy, motive generalization, retrieval refinement, and learned dispatch. Compare one change at a time before a combined configuration. Report paired per-task differences and uncertainty intervals for aggregate rates; do not infer significance from a small change in a single run’s solved count.

13.2 Adversarial regression cases

A correctness-oriented test suite should contain cases that easy examples rarely exercise. Shared expression or universe assignments must invalidate naive per-goal independence. Rollback must restore observations and local-instance behavior. Partial rows with unknown entries must not be transitively merged. A finite prefix of generated carrier values must not be confused with a cardinality theorem.

For recursion, include two occurrences of the same angelic call, a fuel-exhausted but terminating candidate, and a well-founded realization needing a propositional equation. For dependent types, include repeated/composite indices, an index family whose head changes after assignment, and valid local proof goals with free variables. For retrieval, include a useful component that is wrong when applied alone, and an instance unavailable before a type hole is assigned but available afterward. For ranking, include functions equal on all current probes but separated by a later higher-order use.

The purpose is not to demand a universal verifier before any experiment. It is to ensure that the distinctions in Definition 2.1 survive contact with implementation. Where a test lacks a general proof, record that limitation instead of changing the claim to fit a successful run.

13.3 Executed finite-model checks

Only the following independent Python checks were executed for this report. They use the standard library and are fully reproducible from `sanity_checks.py`; the machine-readable output is `sanity_results.json`.

Check	Result
Nine-sample incompatibility graph	Four vertices, six edges: K_4 .
One-state right-fold machines	All 2 accounted for; none consistent.
Two-state right-fold machines	All 128 accounted for; none consistent.
Three-state right-fold machines	All 17,496 accounted for; none consistent.
Four-state witness	Agrees with the target on all 511 words of length at most 8.
Two-feature cover	Both “seen a ” and “seen b ” required in the declared feature library.
Partial observation rows	Compatibility is nontransitive in the displayed counterexample.
Shared-goal cost	True cost 1; sum of two individual lower bounds 2; maximum 1.
Two reverse representations	Both agree with reversal on the same 511 words.
Same-call angel consistency	Independent unequal Boolean answers exist; a shared call cannot satisfy the inequality.
Finite-probe candidate equality	Agreement at 0 does not imply agreement at 1.

The machine enumeration is exact over its declared finite model. For k labeled states, there are k seeds, k^{2k} transition tables over a two-letter alphabet, and 2^k Boolean finishers. The script enumerates every seed/transition pair and counts all consistent finishers from state-output constraints, thereby accounting for

$$k^{2k+1}2^k$$

full machines without needlessly iterating over finishers already contradicted by a shared state. It does not enumerate Lean terms, prove a general minimum carrier-type theorem, or estimate Leant2 runtime.

13.4 Unmeasured outcomes that remain open

The report does not establish the speed of persistent best-first Leant2 states, the benefit of a new residual cache, end-to-end indexed-program coverage, the quality of a carrier predictor, the cost of the pinned `cbv` path on this corpus, or human preference for the proposed ranking. It also does not supply a cross-system performance comparison with `sauto`, although its official reduction and unfolding options were inspected. Each is answered with a specific design or experiment above, not with an unsupported empirical claim.

What *is* established here is narrower and useful: the exact expert-question inventory; source-level integration points and restrictions; several corrections to the proposal’s literature assumptions; the logical conditions for safe pruning, caching, and bounded completeness; and the executed finite countermodels and carrier example. These supply a concrete basis for implementation without presenting a research design as a measured product improvement.

14 Conclusion

Leant2’s central difficulty is not simply that its search language is too small. The next improvements introduce new kinds of guesses—carriers, motives, recursive schemes, library components, proof plans, and learned hints—whose failure modes must not become unjustified logical conclusions. The acceptance gate already provides a valuable final check. The missing

complementary boundary is a precise account of what may safely remove alternatives before that gate is reached.

The proposed development path is therefore evidence-first: explicit persistent state, dependency-correct observations, contextual proof obligations, and release-pinned elaborator/evaluator adapters. Carrier invention can then be attacked constructively through state separation, typed feature selection, and recursive closure. Indexed synthesis can start from existing public motive machinery rather than a new elaborator. Library retrieval can extend established dictionary-aware abstraction refinement while preserving dependent correlations. Learned guidance can remain optional, useful, and auditable without becoming part of the proof foundation.

The result is not a claim that all 31 questions have universally optimal algorithms. It is a set of explicit answers: existing solutions where the literature supplies them, proved limits where stronger guarantees are impossible or unjustified, concrete bounded algorithms where a practical fragment is available, and reproducible studies where the answer must be empirical. That is the appropriate foundation for a synthesizer whose output is intended to be both useful Lean code and reliable evidence.

A Question-to-answer coverage map

The source filename is relative to the proposal’s `sections/` directory. Each identifier links to its full answer. Page numbers refer to this report, not to the original proposal.

ID	Principal answer	Source file	Page
R1.1	Dependency hypergraphs, noninterference, checked memoized schemes.	05-search.tex	6
R1.2	Zero or maximum of justified bounds; additive costs need a proof.	05-search.tex	7
R1.3	Separate fixed ranking costs from adaptive dispatch priorities.	05-search.tex	8
R1.4	Wall-clock obstruction; deterministic logical prefixes and replay.	05-search.tex	8
R2.1	Bounded minimality, incompatibility-graph lower bounds, counterexamples.	06-carriers.tex	9
R2.2	Positional theorem under explicit naturality; equality requires evidence.	06-carriers.tex	11
R2.3	AutoLifter connection; feature separation plus closure refinement.	06-carriers.tex	12
R2.4	Bounded motive grammar with parameter and result generalization.	06-carriers.tex	13
R2.5	Failure-directed invariants and conjecture-test-prove lemma discovery.	06-carriers.tex	14
R3.1	Smyth correction; typed dependent closures and certified observations.	07-evaluation.tex	14
R3.2	From-scratch consistency, immutable versions, rollback dependencies.	07-evaluation.tex	15
R3.3	Existing proof-producing evaluation before a custom certified fragment.	07-evaluation.tex	16
R3.4	Typed shared angels and an explicit bounded completeness theorem.	07-evaluation.tex	17

ID	Principal answer	Source file	Page
R4.1	Finite occurrence templates are not closed under new accumulators.	08-dependent.tex	18
R4.2	Typed index normalization with evidence, not blanket cast erasure.	08-dependent.tex	19
R4.3	Selective unfolding; Canonical and sauto policy evidence.	08-dependent.tex	19
R4.4	Pinned public motive helpers and structured trial diagnostics.	08-dependent.tex	20
R5.1	Propositional interpreter–realizer simulation, not universal conversion.	09-recursion.tex	21
R5.2	Top-down angel constraints with explicit sharing and scoped nogoods.	09-recursion.tex	22
R5.3	A small engineering portfolio; measure expressibility and discovery.	09-recursion.tex	22
R6.1	Dependency-aware value/proof scheduling and local subtype motives.	10-contracts.tex	23
R6.2	A recognizable sufficient proof-plan class with proof reconstruction.	10-contracts.tex	24
R6.3	Existing induction/automation APIs; bounded lemma discovery adaptation.	10-contracts.tex	25
R6.4	Cost-sensitive proof quanta, fair reserve, end-to-end certified payoff.	10-contracts.tex	25
R7.1	Dependent typed hypergraphs and conservative abstraction refinement.	11-retrieval.tex	26
R7.2	Hoogole+ already handles dictionaries; adapt to Lean-specific context.	11-retrieval.tex	27
R7.3	Examples-aware data retrieval and leakage-controlled definition corpora.	11-retrieval.tex	27
R8.1	Frozen MDL-style preference, explicit policy, and human evaluation.	12-ranking.tex	28
R8.2	Separate checked equality from finite-probe presentation buckets.	12-ranking.tex	29
R9.1	No imported accuracy claim; isolated proposal and certification study.	12-ranking.tex	29
R9.2	Complete proposal receipts, controlled syntax, fresh independent checks.	12-ranking.tex	30

B Reproduction and source notes

The bundle contains the self-contained `article.tex`, compiled `article.pdf`, this report's Python reference checks and their output, a source manifest, a question index, and a README. Bibliography entries use immutable repository links where source-version assertions matter. Links to current official documentation are marked as accessed on 21 September 2026 and are not silently equated with the pinned release. Paper versions are specified where their development history affects the answer.

Build the article with two or more passes of pdfLaTeX:

```
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
python3 sanity_checks.py --output sanity_results.json
```

The source requires ordinary LaTeX packages listed in its preamble and Python 3.10 or later for the checks. It does not download or run repository code. No external bibliography processor is needed. The mathematical pseudocode is not presented as compilable Lean, and no untested Lean file is included as though it had passed a kernel check.

The source review used public GitHub content through the available connector and public primary research/official documentation. A local clone was not available in the execution environment, and Lean was not installed there. Those limits affect reproducible performance and integration claims, not the ability to inspect the cited source or execute the independent finite checks. The repository was not modified.

References

- [1] Vladimir Reshetnikov. *Leant2*. Repository snapshot, 18 September 2026.
784664ff34e819422510a4d470ab07fe48bb736b. <https://github.com/VladimirReshetnikov/Leant2/tree/784664ff34e819422510a4d470ab07fe48bb736b>.
- [2] Leant2 project. *Leant2: Design for Further Improvements*. 18 September 2026. Main source; distinguishes original measurements from proposals.
<https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/docs/proposals/11-further-improvements/Leant2-next.tex>.
- [3] Leant2 project. R1: Search control with coupled goals under a wall-clock budget.
<https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/docs/proposals/11-further-improvements/sections/05-search.tex>.
- [4] Leant2 project. R2: Carriers, accumulators and auxiliary state.
<https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/docs/proposals/11-further-improvements/sections/06-carriers.tex>.
- [5] Leant2 project. R3: Evaluation of programs with holes.
<https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/docs/proposals/11-further-improvements/sections/07-evaluation.tex>.
- [6] Leant2 project. R4: Indexed families and the motive problem.
<https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/docs/proposals/11-further-improvements/sections/08-dependent.tex>.
- [7] Leant2 project. R5: General recursion from incomplete examples.
<https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/docs/proposals/11-further-improvements/sections/09-recursion.tex>.
- [8] Leant2 project. R6: Contracts beyond decidable observations.
<https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/docs/proposals/11-further-improvements/sections/10-contracts.tex>.
- [9] Leant2 project. R7: Component retrieval at library scale.
<https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/docs/proposals/11-further-improvements/sections/11-retrieval.tex>.
- [10] Leant2 project. R8: Ranking and candidate equivalence; R9: Learned guidance under a kernel gate.
<https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/docs/proposals/11-further-improvements/sections/12-ranking.tex>.
- [11] Leant2 project. *Leant2/Accept/Gate.lean*. In particular, *kernelCheckAndAudit* and *gate*.
<https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/Leant2/Accept/Gate.lean>.
- [12] Leant2 project. *Leant2/Search/Core.lean*. Inspected portions include lines 1–470: branch state, residual evaluation, proof portfolio, and structural recursion.
<https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/Leant2/Search/Core.lean>.
- [13] Leant2 project. *Leant2/Engine.lean*. Inspected enumeration, lanes, candidate extraction, ranking, and gate calls. <https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/Leant2/Engine.lean>.
- [14] Leant2 project. *lean-toolchain*. <https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/lean-toolchain>.

- [15] Jannis Limperg and Asta Halkjær From. Aesop: White-Box Best-First Proof Search for Lean. *CPP*, pp. 253–266, 2023. DOI: <https://doi.org/10.1145/3573105.3575671>. Primary conference record: <https://popl23.sigplan.org/details/CPP-2023-papers/5/Aesop-White-Box-Best-First-Proof-Search-for-Lean>.
- [16] Chase Norman and Jeremy Avigad. *Canonical for Automated Theorem Proving in Lean*. ITP 2025; arXiv:2504.06239. Version 1 inspected. <https://arxiv.org/html/2504.06239v1>.
- [17] Shraddha Barke, Hila Peleg, and Nadia Polikarpova. Just-in-Time Learning for Bottom-Up Enumerative Synthesis. *OOPSLA*, 2020. arXiv:2010.08663. <https://arxiv.org/abs/2010.08663>.
- [18] Michael Luby, Alistair Sinclair, and David Zuckerman. Optimal Speedup of Las Vegas Algorithms. *Information Processing Letters* 47, pp. 173–180, 1993. [https://doi.org/10.1016/0020-0190\(93\)90029-9](https://doi.org/10.1016/0020-0190(93)90029-9).
- [19] Jeremy Gibbons, Graham Hutton, and Thorsten Altenkirch. When is a Function a Fold or an Unfold? *Electronic Notes in Theoretical Computer Science* 44(1), 2001. Author/institutional publication record: <https://www.cs.ox.ac.uk/publications/publication2368-abstract.html>.
- [20] Dana Angluin. Learning Regular Sets from Queries and Counterexamples. *Information and Computation* 75, pp. 87–106, 1987. [https://doi.org/10.1016/0890-5401\(87\)90052-6](https://doi.org/10.1016/0890-5401(87)90052-6).
- [21] Ruyi Ji, Yuwei Zhao, Yingfei Xiong, Di Wang, Lu Zhang, and Zhenjiang Hu. *Decomposition-Based Synthesis for Applying Divide-and-Conquer-Like Algorithmic Paradigms*. arXiv:2202.12193, version 4, 29 January 2024; paper reports acceptance to ACM TOPLAS. <https://arxiv.org/html/2202.12193v4>.
- [22] Koen Claessen, Moa Johansson, Dan Rosén, and Nicholas Smallbone. HipSpec: Automating Inductive Proofs of Program Properties. ATx 2012 workshop paper, published 2013. DOI: <https://doi.org/10.29007/3qwr>. Author copy: <https://smallbone.se/papers/hipspec-atx.pdf>.
- [23] Justin Lubin, Nick Collins, Cyrus Omar, and Ravi Chugh. Program Sketching with Live Bidirectional Evaluation. *Proceedings of the ACM on Programming Languages* 4(ICFP), article 109, 2020. DOI: <https://doi.org/10.1145/3408991>. <https://arxiv.org/abs/1911.00583>.
- [24] Anders Miltner, Adrian Trejo Nuñez, Ana Brendel, Swarat Chaudhuri, and Isil Dillig. Bottom-up Synthesis of Recursive Functional Programs using Angelic Execution. *POPL*, 2022; arXiv:2107.06253. <https://arxiv.org/abs/2107.06253>.
- [25] Matthew A. Hammer, Jana Dunfield, Kyle Headley, Nicholas Labich, Jeffrey S. Foster, Michael Hicks, and David Van Horn. *Incremental Computation with Names*. 2015. arXiv:1503.07792. <https://arxiv.org/abs/1503.07792>.
- [26] Mario Carneiro. *Lean4Lean: Towards a Verified Typechecker for Lean, in Lean*. 2024. arXiv:2403.14064. <https://arxiv.org/abs/2403.14064>.
- [27] Lean4Lean project. *Lean4Lean: Mechanizing the Metatheory of Lean*. WITS 2026 conference presentation record. <https://popl26.sigplan.org/details/wits-2026-papers/11/-Lean4Lean-Mechanizing-the-Metatheory-of-Lean>.
- [28] Lean project. `src/Lean/Meta/Tactic/Induction.lean`, tag v4.34.0. Public induction and motive-construction implementation. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/Induction.lean>.
- [29] Lean project. `src/Lean/Elab/Tactic/Cbv.lean`, tag v4.34.0. `cbv` and `decide_cbv` tactic adapters. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Elab/Tactic/Cbv.lean>.
- [30] Lean project. *Lean Language Reference: Tactic Reference*. Current online documentation, accessed 21 September 2026. <https://lean-lang.org/doc/reference/latest/Tactic-Proofs/Tactic-Reference/>.

- [31] Lean project. *Lean Language Reference: Recursive Definitions*. Current online documentation, accessed 21 September 2026. <https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/>.
- [32] Lean project. *Theorem Proving in Lean 4: Inductive Types*, especially the natural-number definitions and induction examples. Current online documentation, accessed 21 September 2026. https://lean-lang.org/theorem_proving_in_lean4/Inductive-Types/.
- [33] Nadia Polikarpova, Ivan Kuraj, and Armando Solar-Lezama. Program Synthesis from Polymorphic Refinement Types. *PLDI*, 2016. DOI: <https://doi.org/10.1145/2908080.2908093>. <https://arxiv.org/abs/1510.08419>.
- [34] Zheng Guo, Michael James, David Justo, Jiaxiao Zhou, Ziteng Wang, Ranjit Jhala, and Nadia Polikarpova. Program Synthesis by Type-Guided Abstraction Refinement. *Proceedings of the ACM on Programming Languages* 4(POPL), article 12, 2020. DOI: <https://doi.org/10.1145/3371080>. Author copy: https://ranjitjhala.github.io/static/hoogle_plus.pdf.
- [35] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. *NeurIPS*, 2023. arXiv:2306.15626. <https://arxiv.org/abs/2306.15626>.
- [36] Thomas Zhu, Joshua Clune, Jeremy Avigad, Albert Qiaochu Jiang, and Sean Welleck. *Premise Selection for a Lean Hammer*. ICLR 2026; arXiv:2506.07477, version 2, 2026. <https://arxiv.org/html/2506.07477v2>. https://proceedings.iclr.cc/paper_files/paper/2026/file/cf6501108fced72ee5c47e2151c4e153-Paper-Conference.pdf.
- [37] CoqHammer project. *CoqHammer: Proof Automation for Dependent Type Theory*. Official documentation, especially Sauto options and Hammer reconstruction guidance. Accessed 21 September 2026. <https://coqhammer.github.io/>.